

Generalized Integrated Interleaved Codes

Yingquan Wu, *Senior Member, IEEE*

Abstract—Generalized integrated interleaved codes refer to two-level Reed–Solomon codes, such that each code of the nested layer belongs to different subcode of the first-layer code. In this paper, we first devise an efficient decoding algorithm by ignoring first-layer miscorrection and by intelligently reusing preceding results during each iteration of a decoding attempt. Neglecting first-layer miscorrection also enables to explicitly and neatly formulate the decoding failure probability. We next derive an erasure correcting algorithm for redundant arrays of independent disks systems. We further construct an algebraic systematic encoding algorithm, which had been open. Analogously, we propose a novel generalized integrated interleaving scheme over binary Bose–Chaudhuri–Hocquenghem codes, reveal a lower bound on the minimum distance, and derive a similar encoding and decoding algorithm as those of Reed–Solomon codes.

Index Terms—Reed–Solomon codes, Bose–Chaudhuri–Hocquenghem (BCH) codes, integrated interleaving, generalized integrated interleaved codes, linear-feedback-shift-register (LFSR) encoding, syndrome decoding, erasure correcting, redundant arrays of independent disks (RAID).

I. INTRODUCTION

TO ACHIEVE better protection over an array of interleaves within a single block, a two-level scheme was introduced in [1] and subsequently generalized in [2] to provide nonuniform redundancy, in which extra check symbols are shared among all the interleaves and used by interleaves with errors beyond their decoding distance. However, this construction does not provide protection to the shared redundancies. Instead, another code is required to protect these shared check symbols from errors. The integrated interleaved (II) coding scheme [3], [4] provides an improvement by creating shared redundancy that is protected by the first-layer code. Specifically, the II coding scheme [3], [4] nests a set of m equally protected interleaves with v ($v < m$) more powerful codewords in the second (interchangeably “nested”) layer which is a subcode of the first layer. Specifically, let $\{C_i(n, k_i, d_i)\}_{i=0}^{m-1}$ be two Reed–Solomon (RS) codes over the Galois field $\text{GF}(q)$ such that $C_1 \subset C_0$. An II code is defined as follows

$$C \triangleq \left\{ \mathbf{c} = [\mathbf{c}_0, \mathbf{c}_1, \mathbf{c}_2, \dots, \mathbf{c}_{m-1}] : \mathbf{c}_i \in C_0, \forall i, \sum_{i=0}^{m-1} \alpha^{bi} \mathbf{c}_i \in C_1, 0 \leq b < v \right\}, \quad (1)$$

Manuscript received November 4, 2015; revised April 27, 2016; accepted November 7, 2016. Date of publication November 29, 2016; date of current version January 18, 2017. This work was presented at the 2016 IEEE International Symposium on Information Theory Proceedings.

The author is with Micron Technology, Santa Clara, CA 95054 USA (e-mail: yingquan_wu@yahoo.com).

Communicated by S. ten Brink, Associate Editor for Communications.

Color versions of one or more of the figures in this paper are available online at <http://ieeexplore.ieee.org>.

Digital Object Identifier 10.1109/TIT.2016.2633257

0018-9448 © 2016 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.

See http://www.ieee.org/publications_standards/publications/rights/index.html for more information.

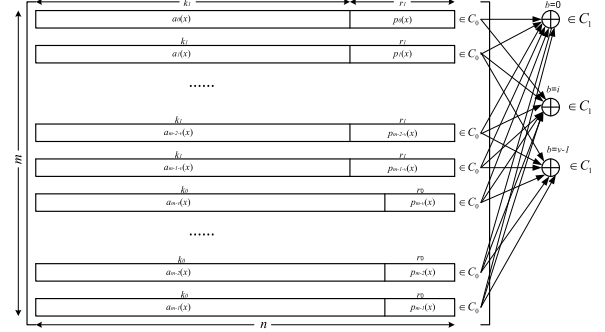


Fig. 1. A pictorial description of II-RS codes.

where $v < m < q$ and α is a primitive element of $\text{GF}(q)$. Its pictorial description is given in Figure 1. Herein an interleave refers to a component word, $\mathbf{c}_i \in C_0$ ($0 \leq i < m$). For a received word $\mathbf{y}$, self-decoding of an interleave $\mathbf{y}_i$, $0 \leq i < m$, refers to the stand-alone decoding of $\mathbf{y}_i$ within the first layer, whereas nested decoding of $\mathbf{y}_i$ refers to the decoding in the nested layer resorting to the entire word $\mathbf{y}$.

Cox *et al.* [3] described an algebraic systematic encoding method for a class of variant (by inserting zeros) II-RS codes with $v = 1$. Wu [5] presented an algebraic systematic encoding algorithm for new variant (by inserting zeros differently) II-RS codes. Hassner *et al.* [4] characterized the general two-layer II codes (with $v \geq 1$), and derived a non-systematic encoding algorithm as well as an algebraic decoding algorithm. Its decoding performance was re-examined and a tight theoretical evaluation was given in [7]. The main complication stems from the miscorrection of the component words. Asano *et al.* [6] considered a special class of three-layer II coding scheme with $v = 1$ at the second and third layers.

Tang and Koetter [8] proposed and characterized a generalized II coding scheme to allow unequal protection in the nested layer, based on the rationale that the strongest code is used to correct the most corrupted interleave while the weakest code corrects the least corrupted interleave. The authors also presented a decoding algorithm which is sophisticated due to handling miscorrection of the interleave self-decoding, and cumbersome due to the large amount of repetitive operations. Moreover, the encoding of GII codes was not investigated in [8].

Let $\{C_i(n, k_i, d_i)\}_{i=0}^v$ be over the Galois field $\text{GF}(q)$ such that

$$C_v \subseteq C_{v-1} \subseteq C_{v-2} \subseteq \dots \subseteq C_1 \subset C_0. \quad (2)$$

TABLE I
AN EXAMPLE OF ADVANCED RAID SYSTEM USING GII-RS SCHEME

D	D	D	D	D	D	D	D	D	D	D	D	D	D	D	P	P
D	D	D	D	D	D	D	D	D	D	D	D	D	D	D	P	P
D	D	D	D	D	D	D	D	D	D	D	D	D	D	P	P	P
D	D	D	D	D	D	D	D	D	D	D	D	D	D	D	D	P
D	D	D	D	D	D	D	D	D	D	D	D	D	D	D	D	P
D	D	D	D	D	D	D	D	D	D	D	D	D	D	D	D	P
D	D	D	D	D	D	D	D	D	D	D	D	D	D	D	D	P
D	D	D	D	D	D	D	D	D	D	D	D	D	D	D	D	P

page is filled before programming. Binary Bose-Chaudhuri-Hocquenghem (BCH) codes have been extensively employed to tackle bit corruptions caused by retention. The proposed GII-BCH scheme does not cause write latency, and moreover, each sector can be read out independently under normal conditions (meaning each sector is correctable by the first layer self-decoding). Another application is to trade off format efficiency with power consumption. A data sector was defined in form of 512-Byte initially, started to shift to 4K-Byte 5-10 years ago, and is gradually moving to 8K-Byte. The format efficiency due to longer code length diminishes after 2K-Byte for prevalent coding schemes, while circuit complexity and power consumption roughly increases linearly with code length. To this end, GII code serves as an alternative to be more complex but also more efficient than simple interleaves, on the other hand, less complex but also less efficient than a single code.

For all potential applications of II/GII codes, it is essential to make the miscorrection negligible for the interleave self-decoding, since it enables to decode each interleave independently under normal operations where the noise level is far below the error correction capability. On the other hand, this is not a stringent condition to meet. For erasure-only decoding (in RAID type architectures), there incurs no miscorrection. For RS or BCH codes with relatively large minimum distances, the miscorrection probability is way below the level of practical tolerance [15]. This enforcement also significantly simplifies the nested decoding and the corresponding performance analysis. We also observe that all existing decoding algorithms proposed for II/GII codes [4], [7], [8] are computationally inefficient, in two perspectives; syndrome computation and key-equation-solver. The existing algorithms always rebuild corrupted nested interleaves and then compute their syndromes. This can be achieved more efficiently by computing syndromes on the constructed nested (corrupted) interleaves only on the first time while updating syndromes by removing contributions from the corrected interleaves at each iteration of a decoding attempt. Moreover, up-to-date syndromes for existing uncorrectable interleaves are valid (but inadequate), therefore, each subsequent attempt will only entail to generate higher order syndromes. Additionally, when the Berlekamp-Massey algorithm is deployed to decode interleaves, their preceding outputs are readily reused in the next iteration by incorporating extra syndromes.

The rest of the paper is organized as follows. Section II introduces the background of II-RS and GII-RS codes, including code dimension, minimum distance and parity check

matrix representation. Section III contains three parts, Part A presents a new efficient decoding algorithm; Part B analyzes decoding performance; Part C derives an erasure correcting algorithm for RAID systems; Part D constructs an efficient systematic encoding algorithm utilizing a linear-feedback-shift-register circuit. Section IV introduces and characterizes a novel GII scheme over binary BCH codes by converting each non-binary weight α^{bi} (in (3)) into its associated binary polynomial, and derives their encoding and decoding algorithm. Section V concludes by drawing pertinent remarks.

II. GENERALIZED INTEGRATED INTERLEAVED REED-SOLOMON CODES

In literature, all studies on (generalized) integrated interleaved (II/GII) coding scheme are carried over Reed-Solomon (RS) codes, due to the existence of efficient syndrome decoder and difficulty to extend to other codes. In [8], the properties of GII-RS codes are stated as follows without proof. Herein we encompass the proof for self-containment

Theorem 1: Let $\{C_i(n, k_i, d_i)\}_{i=0}^v$ be RS codes over the Galois field $GF(q)$ such that

$$\begin{aligned} C_{i_s} &= \dots = C_{i_{s-1}+1} \subset C_{i_{s-1}} = \dots = C_{i_{s-2}+1} \\ &\subset \dots \subset C_{i_1+1} = \dots = C_1 \subset C_0 \end{aligned} \quad (5)$$

where $i_0 = 0$ and $i_s = v$. Let the GII-RS code, $GII([m, v], n, [d_0, d_1, \dots, d_v])$, be defined as in (3), where the minimum distance sequence $[d_0, d_1, \dots, d_v]$ follows the non-decreasing order. The GII-RS code is a linear block code over $GF(q)$ of length $N = mn$, dimension $K = \sum_{i=1}^v k_i + (m - v)k_0$, and minimum distance

$$d_{\min} = \min \{(v+1)d_0, (v-i_1+1)d_{i_1}, \dots, (v-i_{s-1}+1)d_{i_{s-1}}, d_v\}. \quad (6)$$

Note that the notation $GII([m, v], n, [d_0, d_1, \dots, d_v])$ completely describes a GII-RS code. However, it is lengthy. In the sequel, we may drop some last terms for conciseness given no ambiguity, e.g., $GII([m, v], n)$, or $GII([m, v])$.

Proof: For a codeword $\mathbf{c} = [c_0, \mathbf{c}_1, \dots, \mathbf{c}_m] \in \mathcal{C}$, we define a mapping word $\mathcal{V}(\mathbf{c}) = [c'_0, \dots, c'_{v-1}, \mathbf{c}_v, \dots, \mathbf{c}_{m-1}]$ such that

$$c'_j = \sum_{i=0}^{m-1} \alpha^{ji} c_i, \quad 0 \leq j < v.$$

The map is more clearly described by

$$\begin{bmatrix} \mathbf{c}'_0 \\ \mathbf{c}'_1 \\ \vdots \\ \mathbf{c}'_{v-1} \end{bmatrix} = \begin{bmatrix} 1 & 1 & \dots & 1 \\ 1 & \alpha & \dots & \alpha^{m-1} \\ \vdots & \vdots & \ddots & \vdots \\ 1 & \alpha^{v-1} & \dots & \alpha^{(m-1)(v-1)} \end{bmatrix} \begin{bmatrix} \mathbf{c}_0 \\ \mathbf{c}_1 \\ \vdots \\ \mathbf{c}_{m-1} \end{bmatrix}.$$

Since the connection matrix is Vandermonde, we have $\mathcal{V}(\mathbf{c}) \neq \mathcal{V}(\mathbf{c}')$ iff $\mathbf{c} \neq \mathbf{c}'$. By definition,

$$\mathbf{c}'_i \in \mathcal{C}_{v-i}, \quad i = 0, 1, 2, \dots, v-1.$$

Therefore, we obtain the size of the code

$$|\mathcal{C}| = |\mathcal{V}(\mathcal{C})| = q^{\sum_{i=1}^v k_i + (m-v)k_0}.$$

We next show that the minimum distance is given by (6). If a codeword $\mathbf{c}$ has at least $v+1$ nonzero interleaves, then its Hamming weight is at least $(v+1)d_0$. On the other hand, let

$$\begin{bmatrix} 1 & 1 & \dots & 1 \\ 1 & \alpha & \dots & \alpha^v \\ \vdots & \vdots & \ddots & \vdots \\ 1 & \alpha^{v-1} & \dots & \alpha^{v(v-1)} \end{bmatrix} \begin{bmatrix} 1 \\ \gamma_1^{(0)} \\ \vdots \\ \gamma_v^{(0)} \end{bmatrix} = \mathbf{0}$$

and $\mathbf{c}_0 \in \mathcal{C}_0$ be with minimum Hamming weight d_0 , then the following codeword lies in $\mathcal{C}$

$$\mathbf{c} = [\mathbf{c}_0, \gamma_1^{(0)}\mathbf{c}_0, \dots, \gamma_v^{(0)}\mathbf{c}_0, \mathbf{0}, \dots, \mathbf{0}]$$

and has exactly $v+1$ nonzero interleaves and Hamming weight $(v+1)d_0$.

When a codeword $\mathbf{c}$ has exactly $v-i_1+1$ nonzero interleaves, we first show that all its nonzero component words must lie in $\mathcal{C}_1$. Without loss of generality, we assume the first $v-i_1+1$ interleaves are nonzero. Let

$$\begin{bmatrix} \mathbf{c}'_0 \\ \mathbf{c}'_1 \\ \vdots \\ \mathbf{c}'_{v-i_1} \end{bmatrix} \triangleq \begin{bmatrix} 1 & 1 & \dots & 1 \\ 1 & \alpha & \dots & \alpha^{v-i_1} \\ \vdots & \vdots & \ddots & \vdots \\ 1 & \alpha^{v-i_1} & \dots & \alpha^{(v-i_1)^2} \end{bmatrix} \begin{bmatrix} \mathbf{c}_0 \\ \mathbf{c}_1 \\ \vdots \\ \mathbf{c}_{v-i_1} \end{bmatrix}. \quad (7)$$

By definition, $\mathbf{c}'_j \in \mathcal{C}_{v-j} \subseteq \mathcal{C}_1$, $j \leq v-i_1$. Since the connection matrix is invertible, $\mathbf{c}_0, \mathbf{c}_1, \dots, \mathbf{c}_{v-i_1}$, which are linear combinations of $\mathbf{c}'_0, \mathbf{c}'_1, \dots, \mathbf{c}'_{v-i_1}$, also lie in $\mathcal{C}_1$. Therefore, its Hamming weight is at least $(v-i_1+1)d_1$. Conversely, let $\gamma_1^{(1)}, \gamma_2^{(1)}, \dots, \gamma_{v-i_1}^{(1)}$, satisfy

$$\begin{bmatrix} 1 & 1 & \dots & 1 \\ 1 & \alpha & \dots & \alpha^{v-i_1} \\ \vdots & \vdots & \ddots & \vdots \\ 1 & \alpha^{v-i_1-1} & \dots & \alpha^{(v-i_1)(v-i_1-1)} \end{bmatrix} \begin{bmatrix} 1 \\ \gamma_1^{(1)} \\ \vdots \\ \gamma_{v-i_1}^{(1)} \end{bmatrix} = \mathbf{0} \quad (8)$$

and $\mathbf{c}_1 \in \mathcal{C}_1$ be with minimum Hamming weight d_1 . It is easy to verify that

$$\mathbf{c} = [\mathbf{c}_1, \gamma_1^{(1)}\mathbf{c}_1, \dots, \gamma_{v-i_1}^{(1)}\mathbf{c}_1, \mathbf{0}, \dots, \mathbf{0}]$$

is a valid codeword in $\mathcal{C}$. Moreover, this codeword has exactly $v-i_1+1$ nonzero interleaves in $\mathcal{C}_1$ and Hamming weight $(v-i_1+1)d_1$. On the other hand, if the number is non-zero interleaves in $\mathcal{C}_1$ is less by 1, then (8) is not enforceable,

consequently $\mathbf{c}'_j$, $j = 0, 1, 2, \dots, v-i_1$, defined in (7), fail to satisfy $\mathbf{c}'_j \in \mathcal{C}_j$. Therefore, $(v-i_1+1)d_1$ is indeed a tight upper bound of the minimum distance. The proof is concluded by induction. $\square\square$

In the most common scenario where the nesting minimum distance sequence satisfies a strictly increasing order,

$$d_0 < d_1 < d_2 < \dots < d_v, \quad (9)$$

the minimum distance of a GII-RS code is neatly expressed in the following corollary.

Corollary 1: Let a GII-RS code $\text{GII}([m, v], n, [d_0, d_1, \dots, d_v])$ be defined as in (3). If the nesting minimum distance sequence satisfies a strictly increasing order as in (9), then the code $\mathcal{C}$ exhibits a minimum distance of $\min\{(v+1)d_0, vd_1, \dots, 2d_{v-1}, d_v\}$.

Theorem 1 trivially characterizes an II-RS code, denoted by $\text{II}([m, v], n, [d_0, d_1])$, as given in [4].

Corollary 2: An II-RS code $\text{II}([m, v], n, [d_0, d_1])$, defined in (1), is a linear block code over $\text{GF}(q)$ of length mn , dimension $(m-v)k_0 + vk_1$, and minimum distance $\min\{(v+1)d_0, d_1\}$. Clearly, GII scheme provides great flexibility in code design and may significantly increase minimum distance, over the original II scheme.

We next show that GII-RS codes share certain similarity with generalized error-location codes (cf. [13]) in terms of parity check matrix composition. Denote by $\mathbf{H}(r, n; f)$ the parity check matrix of a Reed-Solomon $\mathcal{C}(n, n-r)$ code with starting spectrum α^f ($f = 0$ is used in subsequent presentation),

$$\mathbf{H}(r, n; f) = \begin{bmatrix} \alpha^{(n-1)f} & \dots & \alpha^f & 1 \\ \alpha^{(n-1)(f+1)} & \dots & \alpha^{f+1} & 1 \\ \vdots & \ddots & \vdots & \vdots \\ \alpha^{(n-1)(f+r-1)} & \dots & \alpha^{f+r-1} & 1 \end{bmatrix}. \quad (10)$$

Note the reverse order of the above matrix is due to systematic encoding such that a codeword is reversely indexed by $[c_{n-1}, c_{n-2}, \dots, c_1, c_0]$. Denote by $\bar{\mathbf{h}}(m, b)$ the coefficient vector

$$\bar{\mathbf{h}}(m, b) = [1, \alpha^b, \alpha^{2b}, \dots, \alpha^{(m-1)b}], \quad (11)$$

and $\mathbf{I}_m$ the $m \times m$ identity matrix. Then the parity matrix of the GII-RS code defined in (3) is given by

$$\mathbf{H}([m, v], n) = \begin{bmatrix} \mathbf{I}_m \otimes \mathbf{H}(r_0, n; f) \\ \bar{\mathbf{h}}(m, 0) \otimes \mathbf{H}(r_v - r_0, n; r_0 + f) \\ \bar{\mathbf{h}}(m, 1) \otimes \mathbf{H}(r_{v-1} - r_0, n; r_0 + f) \\ \bar{\mathbf{h}}(m, 2) \otimes \mathbf{H}(r_{v-2} - r_0, n; r_0 + f) \\ \vdots \\ \bar{\mathbf{h}}(m, v-1) \otimes \mathbf{H}(r_1 - r_0, n; r_0 + f) \end{bmatrix} \quad (12)$$

where $\otimes$ denotes the Kronecker product, and f is the starting spectrum index of the RS code $\mathcal{C}_0$. Clearly, GII-RS codes share the composition of the Kronecker products with generalized error-location (GEL) codes, which are equivalent to generalized concatenated codes [13]. However, their inner codes are formed differently. In GII-RS codes, the inner codes are composed of Reed-Solomon codes of the same length and

moreover form a subset sequence (2), whereas inner codes have different length and together form a parity check matrix in semi-lower triangular shape [13, eq. (9)] in GEL codes.

A. Hard Decoding Algorithm

The decoding algorithm for GII-RS codes and its analysis proposed in [8] is significantly convolved due to handling possible miscorrection of interleaves. In fact, a key feature of the integrated-interleaved codes is that it enables to decode each interleave independently during normal operation, whereas the second layer decoding is only provoked if one or more interleaves rarely suffer self-decoding failure. This feature is essential for distributed storage and power saving. Indeed, for reasonably large minimum distance (say, $d_0 > 10$) of practically deployed Reed-Solomon codes, its miscorrection probability is far below our acceptance criterion [15]. To this end, we shall neglect miscorrection for interleave self-decoding, rendering a much simpler decoding algorithm.

Moreover, the decoding algorithm in [8] is computationally cumbersome, in the sense that the outputs of failed decoding attempts are simply discarded, resulting in a great deal of repeated computation. In this section, we show that syndromes for failed decoding attempts can be reused for future decoding attempts, and furthermore, by using the Berlekamp-Massey algorithm, the error locator polynomial and its auxiliary polynomial can be reused to incrementally generate new ones in next round of decoding attempt. Before demonstrating our new decoding algorithm, we briefly introduce the Berlekamp-Massey algorithm and its characteristics.

Consider a (possibly shortened) Reed-Solomon $\mathcal{C}(n, k)$ code in $\text{GF}(q)$ ($n < q$) with generator polynomial

$$g(x) = (x - 1)(x - \alpha)(x - \alpha^2) \dots (x - \alpha^{2t-1}) \quad (13)$$

where $n - k = 2t$ (t is the so-called *error correction capability*), and α is a primitive element of $\text{GF}(q)$. Define codeword polynomial $c(x) \triangleq c_{n-1}x^{n-1} + c_{n-2}x^{n-2} + \dots + c_1x + c_0$ associated with a codeword $\mathbf{c} = [c_{n-1}, c_{n-2}, \dots, c_1, c_0]$ (note that indexes are reverse ordered due to systematic encoding). In the sequel, we will not distinguish a codeword vector $\mathbf{c}$ from its corresponding polynomial $c(x)$.

Let $c(x)$ denote a transmitted codeword polynomial and $y(x)$ the received word polynomial after appropriate channel quantization. Denote by e the (unknown) number of errors, $X_1, X_2, \dots, X_e$ error locators, and $Y_1, Y_2, \dots, Y_e$ the corresponding error magnitudes. Its syndromes are defined as

$$S_i \triangleq \sum_{j=1}^e Y_j X_j^i, \quad i = 0, 1, 2, \dots \quad (14)$$

By definition, a codeword $c(x)$ satisfies $c(\alpha^i) = 0$, $i = 0, 1, \dots, 2t - 1$. Therefore, $\{S_i\}_{i=0}^{2t-1}$ can be computed through evaluating the received word $y(x)$

$$S_i = y(\alpha^i) - c(\alpha^i) = y(\alpha^i), \quad i = 0, 1, 2, \dots, 2t - 1.$$

If all $2t$ syndrome values are zero, then $y(x)$ is a codeword and thus is presumed that $c(x) = y(x)$, i.e., no error has occurred.

Define the *syndrome polynomial*

$$S(x) \triangleq S_0 + S_1x + S_2x^2 + \dots + S_{2t-1}x^{2t-1}. \quad (15)$$

Define the (unknown) *error locator* and *error evaluator* polynomials

$$\Lambda(x) \triangleq \prod_{i=1}^e (1 - X_i x) = 1 + \Lambda_1 x + \dots + \Lambda_e x^e. \quad (16)$$

$$\begin{aligned} \Omega(x) &\triangleq \sum_{i=1}^e Y_i \prod_{j=1, j \neq i}^e (1 - X_j x) \\ &= \Omega_0 + \Omega_1 x + \Omega_2 x^2 + \dots + \Omega_{e-1} x^{e-1}. \end{aligned} \quad (17)$$

The three polynomials satisfy the following *key equation* [16]

$$\Omega(x) = \Lambda(x)S(x) \pmod{x^{2t}}. \quad (18)$$

The Berlekamp-Massey algorithm solves the above key equation with $O(t^2)$ operations, given that the number of errors $e \leq t$. The essence of the Berlekamp-Massey algorithm is to determine a minimum-length linear feedback shift register (LFSR) that *generates* the syndrome sequence $S_0, S_1, S_2, \dots, S_{2t-1}$, where an LFSR of length L , $\Lambda_0 = 1, \Lambda_1, \Lambda_2, \dots, \Lambda_L$, *generates* the sequence $S_0, S_1, S_2, \dots, S_{2t-1}$ if

$$S_i = - \sum_{j=1}^L S_{i-j} \Lambda_j, \quad i = L, L+1, \dots, 2t-1. \quad (19)$$

Once given the error locator polynomial $\Lambda(x)$ along with its length L_Λ by the Berlekamp-Massey algorithm, the error locators along with corresponding error locations are obtained through exhaustive search over the valid root sequence $1, \alpha^{-1}, \alpha^{-2}, \dots, \alpha^{-(n-1)}$. $\Lambda(x)$ yields a valid codeword iff its number of distinct roots within $\{\alpha^{-i}\}_{i=0}^{n-1}$ equals its length L_Λ . The Horiguchi-Koetter formula [16, p. 225] directly utilizes $\Lambda(x)$ and $B(x)$ generated by the Berlekamp-Massey algorithm to evaluate error magnitudes,

$$Y_i = \frac{X_i^{-(2t-2)}}{B(X_i^{-1})\Lambda'(X_i^{-1})}, \quad i = 1, 2, \dots, e, \quad (20)$$

where $\Lambda'(x)$ is the formal derivative of $\Lambda(x)$.

In II/GII decoding schemes, when an interleave is successfully corrected, its higher order syndromes are computed in an attempt to correct the failed interleaves [4], [8]. In existing decoding algorithms, those syndromes are computed from the scratch. The following lemma demonstrates an alternative, and much more efficient approach [16, p. 183].

Lemma 1: Let $\mathbf{c}$ be a transmitted codeword and $\mathbf{y}$ the received word. If there are $e \leq t$ errors and let $\Lambda(x)$ be the corresponding error locator polynomial, then the higher order syndromes $S_i \triangleq y(\alpha^i) - c(\alpha^i)$, $i \geq 2t$ can be computed recursively through the following LFSR

$$S_i = -\Lambda_1 S_{i-1} - \Lambda_2 S_{i-2} - \dots - \Lambda_e S_{i-e}, \quad i \geq 2t. \quad (21)$$

In II/GII decoding schemes, when an interleave decoding attempt is failed, its higher order syndromes are computed in next round to enforce a stronger correction capability [4], [8]. In existing decoding algorithms,

key-equation-solver, such as the Berlekamp-Massey algorithm, is re-carried each time. The Berlekamp-Massey algorithm recursively computes the minimum-length LFSR polynomial, corresponding to the error-locator polynomial, by incrementally incorporating higher order syndromes. Next Berlekamp-Massey updating algorithm incrementally updates the error locator polynomial utilizing the preceding results and newly available higher order syndromes.

Berlekamp-Massey Updating Algorithm

- Input: $[S_0, \dots, S_{2t-1}, \overbrace{S_{2t}, \dots, S_{2t'-1}}^{\text{new}}, \Lambda^{(2t)}(x), B^{(2t)}(x), L_{\Lambda}^{(2t)}, L_B^{(2t)}$
- For $r = 2t, 2t+1, 2t+2, \dots, 2t'-1$, do:
 - Compute $\Delta^{(r)} = \sum_{i=0}^{L_{\Lambda}} \Lambda_i^{(r)} \cdot S_{r-i}$
 - Compute $\Lambda^{(r+1)}(x) = \Lambda^{(r)}(x) - \Delta^{(r)} \cdot x B^{(r)}(x)$
 - If $\Delta^{(r)} \neq 0$ and $L_{\Lambda}^{(r)} \leq L_B^{(r)}$, then
 - * Set $B^{(r+1)}(x) \leftarrow (\Delta^{(r)})^{-1} \cdot \Lambda^{(r)}(x)$
 - * Set $L_{\Lambda}^{(r+1)} \leftarrow L_{\Lambda}^{(r)} + 1, L_B^{(r+1)} \leftarrow L_{\Lambda}^{(r)}$
 - Else
 - * Set $B^{(r+1)}(x) \leftarrow x B^{(r)}(x)$
 - * Set $L_{\Lambda}^{(r+1)} \leftarrow L_{\Lambda}^{(r)}, L_B^{(r+1)} \leftarrow L_B^{(r)} + 1$
- Outputs: $\Lambda^{(2t')}(x), B^{(2t')}(x), L_{\Lambda}^{(2t')}, L_B^{(2t')}$

We proceed to present our new decoding algorithm while justifying it afterward. For performance efficiency, let code redundancies, $r_0, r_1, \dots, r_v$, be all even numbers, i.e., $r_i = 2t_i$, $i = 0, 1, \dots, v$. The main idea behind decoding is to iteratively correct interleaves with the least number of errors.

Hard Decoding Algorithm for GII-RS Codes

Input: $y(x) = [y_0(x), y_1(x), \dots, y_{m-1}(x)]$

- 1) Compute syndromes $\{S_j^{(i)}\}_{j=0}^{2t_0-1}, i = 0, 1, \dots, m-1$ (where the superscript (i) denotes interleave index i).
- 2) Apply the Berlekamp-Massey algorithm with input $\{S_j^{(i)}\}_{j=0}^{2t_0-1}$ to produce error locator polynomial set $(\Lambda^{(i,2t_0)}(x), B^{(i,2t_0)}(x))$, for $i = 0, 1, \dots, m-1$, respectively. For $i = 0, 1, \dots, m-1$, if $\Lambda^{(i,2t_0)}(x)$ has exactly $L_{\Lambda}^{(i,2t_0)}$ distinct roots within $\{\alpha^{-i}\}_{i=0}^{n-1}$, then correct the interleave y_i , else save the result $(\Lambda^{(i,2t_0)}(x), B^{(i,2t_0)}(x))$.
- 3) Let I be the index set of the uncorrectable interleaves, $I = \{l_1, l_2, \dots, l_b\}$ ($b = |I|$). If $I = \emptyset$, then return the corrected codeword $c(x) = [c_0(x), c_1(x), \dots, c_{m-1}(x)]$; else if $b > v$, then declare failure.

- 4) Compute corrupted nested interleaves

$$\tilde{y}_i(x) = \sum_{j \in I} \alpha^{ij} y_j(x) + \sum_{j \in I^c} \alpha^{ij} c_j(x) \quad (22)$$

for $i = 0, 1, 2, \dots, b-1$.

- 5) Compute higher order syndromes $\{\tilde{S}_j^{(i)}\}_{j=2t_0}^{2t_{v-i}-1}$ over each nested interleaves $\tilde{y}_i(x)$, $i = 0, 1, \dots, b-1$ (Note different number of syndromes are computed over each nested interleave).
- 6) While $b > 0$, do:
 - a) Compute syndromes $\{S_j^{(l_i)}\}_{j=2t_{v-b'}}^{2t_{v-b}-1}, i = 1, 2, \dots, b$, through (23), where b' is initialized to v .
 - b) Apply the Berlekamp-Massey updating algorithm with new inputs $\{S_j^{(l_i)}\}_{j=2t_{v-b'}}^{2t_{v-b}-1}$ to produce new error locator polynomial set $(\Lambda^{(l_i,2t_{v-b})}(x), B^{(l_i,2t_{v-b})}(x))$, for $i = 1, 2, \dots, b$, respectively. For $i = 1, 2, \dots, b$, if $\Lambda^{(l_i,2t_{v-b})}(x)$ has exactly $L_{\Lambda}^{(l_i,2t_{v-b})}$ distinct roots within $\{\alpha^{-i}\}_{i=0}^{n-1}$, then correct the interleave y_{l_i} ; else save the result $(\Lambda^{(l_i,2t_{v-b})}(x), B^{(l_i,2t_{v-b})}(x))$.
 - c) Let I' be the index set of newly corrected interleaves. Set $I \leftarrow I - I'$, $b' \leftarrow b$, and $b \leftarrow |I|$. If $I' = \emptyset$ (no interleave is newly corrected) or $t_{v-b} = t_{v-b'}$ (it fails to produce extra higher order syndromes) then declare failure. If $I = \emptyset$ (all remaining interleaves are corrected) then return the corrected codeword.
 - d) For each $i \in I'$, compute higher order syndromes $\{S_j^{(i)}\}_{j=2t_{v-b'}}^{2t_{v-b}-1}$ through the recursive LFSR (21). Update higher order nested syndromes through

$$\left\{ \tilde{S}_j^{(l)} \leftarrow \tilde{S}_j^{(l)} - \sum_{i \in I'} \alpha^{il} S_j^{(i)} \right\}_{j=2t_{v-b'}}^{2t_{v-l}-1}$$

for $0 \leq l < b$.

It is worth noting that, for single j entry, Step 6.(a) is equivalent to erasure-only decoding for an RS code, thus can be more efficiently implemented by Forney's formula [16, p. 196]. However, v is typically a small integer, say, $v \leq 4$, therefore, the matrix inversion in (23), as shown at the bottom of this page does not need large amount of computation. The block diagram of the above decoding process for the most common case, $\mathcal{C}_v \subset \mathcal{C}_{v-1} \subset \dots \subset \mathcal{C}_1 \subset \mathcal{C}_0$, is illustrated in Figure 3.

Comparing to the original definition in [8], the proposed new definition results in the following decoder simplifications. (i). $\tilde{y}_0(x)$ is always computed in Step 4, with all-one

$$\begin{bmatrix} S_j^{(l_1)} \\ S_j^{(l_2)} \\ \vdots \\ S_j^{(l_b)} \end{bmatrix}_{j=2t_{v-b'}}^{2t_{v-b}-1} = \begin{bmatrix} 1 & 1 & \dots & 1 \\ \alpha^{l_1} & \alpha^{l_2} & \dots & \alpha^{l_b} \\ \vdots & \vdots & \ddots & \vdots \\ \alpha^{(b-1)l_1} & \alpha^{(b-1)l_2} & \dots & \alpha^{(b-1)l_b} \end{bmatrix}^{-1} \begin{bmatrix} \tilde{S}_j^{(0)} \\ \tilde{S}_j^{(1)} \\ \vdots \\ \tilde{S}_j^{(b-1)} \end{bmatrix}_{j=2t_{v-b'}}^{2t_{v-b}-1} \quad (23)$$

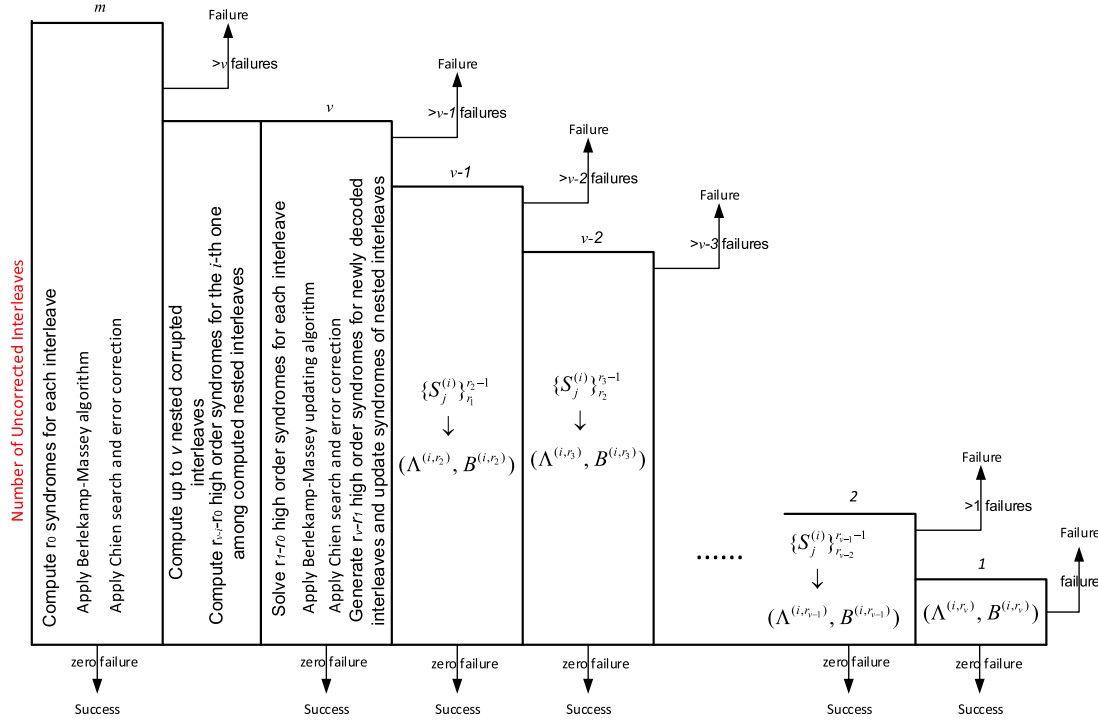


Fig. 3. The block diagram of the proposed GII decoder.

coefficients, i.e., $\tilde{y}_0(x) = \sum_{j \in I} y_j(x) + \sum_{j \in I^c} c_j(x)$. (ii). In Step 6.(a), the connection matrix has all-one first row, which simplifies matrix inversion. (iii). In Step 6.(d), syndrome update for $l = 0$ is simplified to

$$\left\{ \tilde{S}_j^{(0)} \leftarrow \tilde{S}_j^{(0)} - \sum_{i \in I'} S_j^{(i)} \right\}_{j=2t_{v-b'}}^{2t_v-1}.$$

Clearly, the decoding algorithm given in [8] can also be trivially simplified by neglecting the first-layer miscorrection. In comparison, the above decoding algorithm exhibits the following computational advantages over the algorithm in [8]. (i). The above algorithm computes the nested-layer words only when nested-layer decoding is invoked (Step 4), whereas the nested layer code are repeatedly computed during the nested-layer decoding process (Step 6) in [8]. (ii). In the above decoding, only higher order nested syndromes are computed before nested-layer decoding is invoked (Step 5), and subsequently higher order nested syndromes are dynamically updated (Step 6.(d)), whereas all nested syndromes are computed over the updated nested words each time (Step 7) in [8]. (iii). In the above algorithm, only higher order syndromes of uncorrected interleaves are solved (Step 6.(a)) and the Berlekamp-Massey updating algorithm is employed to reuse the preceding outputs to obtain the desired error locator polynomial (Step 6.(b)), whereas all syndromes are solved (Step 8) and the Berlekamp-Massey algorithm is deployed to produce the desired error locator polynomial from scratch (Step 9) in [8].

Theorem 2: By neglecting miscorrection of interleave self-decoding, let $e_0, e_1, \dots, e_{m-1}$ denote the number of errors over received interleaves $y_0(x), y_1(x), \dots, y_{m-1}(x)$, respectively.

Reorder $e_0, e_1, \dots, e_{m-1}$, to $\tau_0, \tau_1, \dots, \tau_{m-1}$, such that

$$\tau_{m-1} \leq \tau_{m-2} \leq \dots \leq \tau_{v+1} \leq \tau_0 \leq \tau_1 \leq \dots \leq \tau_v. \quad (24)$$

The decoding is successful iff

$$\tau_i \leq t_i, \quad i = 0, 1, 2, \dots, v. \quad (25)$$

Proof: It suffices to prove for the worst case scenario,

$$\tau_i = \begin{cases} t_0, & v < i < m \\ t_i, & i \leq v. \end{cases}$$

At the first layer decoding, $m - v$ interleaves with t_0 errors are decoded successfully, but all others (each with more than t_0 errors) are failed by assuming free of miscorrection. We proceed to determine higher order syndromes for uncorrected interleaves. It suffices to show for the first set, $\{S_j\}_{j=2t_0}^{2t_1-1}$. Note that

$$\begin{aligned} \tilde{S}_l^{(i)} &= \tilde{y}_i(\alpha^l) \\ &= \sum_{j \in I} \alpha^{ij} y_j(\alpha^l) + \sum_{j \in I^c} \alpha^{ij} c_j(\alpha^l) \\ &= \sum_{j \in I} \alpha^{ij} (y_j(\alpha^l) - c_j(\alpha^l)) + \sum_{j=0}^{m-1} \alpha^{ij} c_j(\alpha^l) \\ &= \sum_{j \in I} \alpha^{ij} S_l^{(j)} \end{aligned}$$

for $i = 0, 1, 2, \dots, b-1$ (in this worst case $b = v$), and $l = 2t_0, 2t_0 + 1, \dots, 2t_1 - 1$. Putting together all equalities

yields the following linear equation systems

$$\begin{bmatrix} 1 & 1 & \dots & 1 \\ \alpha^{l_1} & \alpha^{l_2} & \dots & \alpha^{l_b} \\ \vdots & \vdots & \ddots & \vdots \\ \alpha^{(b-1)l_1} & \alpha^{(b-1)l_2} & \dots & \alpha^{(b-1)l_b} \end{bmatrix} \cdot \begin{bmatrix} S_j^{(l_1)} \\ S_j^{(l_2)} \\ \vdots \\ S_j^{(l_b)} \end{bmatrix}_{j=2t_{v-b'}}^{2t_{v-b}-1} = \begin{bmatrix} \tilde{S}_j^{(0)} \\ \tilde{S}_j^{(1)} \\ \vdots \\ \tilde{S}_j^{(b-1)} \end{bmatrix}_{j=2t_{v-b'}}^{2t_{v-b}-1}$$

Evidently, the above connection matrix is a Vandermonde matrix and thus syndromes $\{S_j^{(l_i)}\}_{j=2t_0}^{2t_1-1}$, $i = 1, 2, \dots, b$ (herein $b = v$) are resolved by Step 6.(a). This further allows to correct i_1 interleaves each with (up to) t_1 errors while all remaining interleaves (each with more than t_1 errors) suffer decoding failure by assuming free of miscorrection. For the i_1 corrected interleaves, their higher order syndromes can be efficiently computed by recursive LFSR (21), by Lemma 1. Finally the nested syndromes associated with remaining uncorrected interleaves are obtained through Step 6.(d). This procedure is repeated to correct i_2 interleaves with (up to) t_2 errors, and continues until the last interleaves with (up to) t_v errors are corrected.

Conversely, assume one term satisfying $\tau_{v-b} > t_{v-b}$. Then, there exist at least b interleaves with more than t_{v-b} errors. However, the number of nested interleaves capable of correcting more than t_{v-b} errors is at most $b-1$, which yields a decoding failure. The theorem follows. $\square\square$

B. Performance Analysis

We proceed to characterize decoding failure probability under a memoryless channel. Let p_s denote the symbol error rate and

$$\phi_w^n(p_s) \triangleq \binom{n}{w} p_s^w (1-p_s)^{n-w} \quad (26)$$

be the probability of having exactly w errors among n symbols.

By Theorem 2, decoding fails if one of the following mutually exclusive events occurs: there exist at least one interleave with more than t_v errors, or, at least $v-i+1$ interleaves whose error weights are above ($>$) t_i but below ($\leq$) t_{i+1} and the remaining interleaves all have error weights below t_i , $i = 0, 1, 2, \dots, v-1$.

Summing up the probabilities associated with the all above events, we obtain explicitly the decoding failure probability shown in (27), as shown at the bottom of this page.

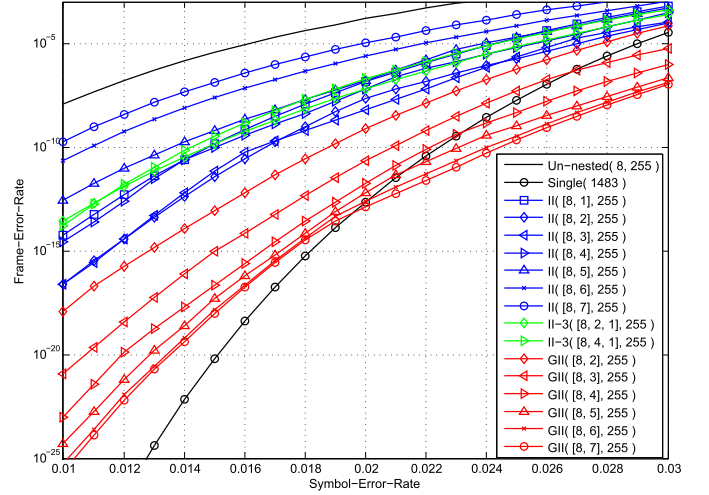


Fig. 4. The performance comparisons among various point-wise optimized II/GII schemes with respect to 8 interleaves.

where the miscorrection in first-layer decoding is not taken into account. Herein we introduce another measurement called invocation probability (denoted by P_i) to indicate the triggering rate of nested decoding. Clearly, nested decoding is triggered when self-decoding of one or more interleaves fails, i.e.,

$$P_i \triangleq 1 - \left(1 - \sum_{w=t_0+1}^n \phi_w^n(p_s)\right)^m. \quad (28)$$

Note that nested decoding costs much more power and longer latency (even more costly if interleaves are distributed at different physical units). Therefore, P_i serves a critical metric for practical design (but is largely neglected by researcher community).

Figure 4 draws comparisons among the hard-decision decoding on 8 interleaved Reed-Solomon (255, 223, 33) codes and various nested coding schemes with equal overall parity length. In the figure, II-3($[m, v, 1], n, [d_1, d_2, d_3]$) denotes a special three-layer II code given in [6], and is formally defined

$$\mathcal{C} \triangleq \left\{ \mathbf{c} = [\mathbf{c}_0, \mathbf{c}_1, \mathbf{c}_2, \dots, \mathbf{c}_{m_1 m_2 - 1}] : \mathbf{c}_i \in \mathcal{C}_0, \forall i, \right. \\ \left. \times \sum_{i=0}^{m_2-1} \mathbf{c}_{j m_1 + i} \in \mathcal{C}_1, 0 \leq j < m_1, \sum_{i=0}^{m_1 m_2 - 1} \mathbf{c}_i \in \mathcal{C}_2 \right\}, \quad (29)$$

where $\{\mathcal{C}_i(n, k_i, d_i)\}_{i=0}^2$ satisfy $\mathcal{C}_2 \subset \mathcal{C}_1 \subset \mathcal{C}_0$. Its main advantage lies in simplicity in both encoder and decoder, due to a single node nesting (i.e., $v = 1$) in the second and third layers. However, its performance is evidently inferior to the counterpart GII code given by

$$P_f = \sum_{b=1}^m \binom{m}{b} \left(\sum_{w=t_v+1}^n \phi_w^n(p_s) \right)^b \left(\sum_{w=0}^{t_v} \phi_w^n(p_s) \right)^{m-b} + \sum_{i=0}^{v-1} \sum_{b=v-i+1}^m \binom{m}{b} \left(\sum_{w=t_{i+1}}^{t_{i+1}} \phi_w^n(p_s) \right)^b \left(\sum_{w=0}^{t_i} \phi_w^n(p_s) \right)^{m-b} \quad (27)$$

TABLE II
PERFORMANCE COMPARISONS OF VARIOUS NESTED RS CODES OPTIMIZED UNDER A SYMBOL-ERROR-RATE OF 0.02

Nesting Scheme	$d_{\min}$	$P_f(0.02)$	$P_i(0.02)$
Un-nested(8, 255, 33)	33	1.55e-4	
II([8, 1], 255, [31, 47])	47	1.37e-7	5.51e-4
II([8, 2], 255, [27, 51])	51	2.14e-8	5.78e-3
II([8, 3], 255, [25, 47])	47	6.09e-9	1.69e-2
II([8, 4], 255, [21, 45])	45	6.01e-8	0.111
II([8, 5], 255, [19, 43])	43	1.58e-7	0.242
II-3([8, 2, 1], 255, [31, 33, 45])	45	6.38e-8	5.51e-4
II-3([8, 4, 1], 255, [29, 33, 49])	49	1.90e-7	1.84e-3
GII([8, 2], 255, [29, 37, 53])	53	7.48e-10	1.84e-3
GII([8, 3], 255, [27, 33, 39, 57])	57	2.09e-11	5.78e-3
GII([8, 4], 255, [25, 29, 35, 41, 59])	59	1.78e-12	1.69e-2
GII([8, 5], 255, [23, 27, 31, 35, 43, 59])	59	5.97e-13	4.54e-2
GII([8, 6], 255, [23, 25, 27, 31, 35, 41, 59])	59	2.53e-13	4.54e-2
GII([8, 7], 255, [21, 23, 25, 27, 31, 35, 43, 59])	59	1.29e-13	0.111

TABLE III
PERFORMANCE COMPARISONS OF VARIOUS NESTED RS CODES OPTIMIZED UNDER A SYMBOL-ERROR-RATE OF 0.01

Nesting Scheme	$d_{\min}$	$P_f(0.01)$	$P_i(0.01)$
Un-nested(8, 255, 33)	33	1.12e-8	
II([8, 1], 255, [31, 47])	47	5.38e-15	7.96e-8
II([8, 2], 255, [27, 51])	51	2.55e-17	3.31e-6
II([8, 3], 255, [23, 51])	51	2.35e-17	1.04e-4
II([8, 4], 255, [19, 47])	47	2.73e-15	2.34e-3
II([8, 5], 255, [19, 43])	43	2.64e-13	2.34e-3
II-3([8, 2, 1], 255, [31, 33, 45])	45	2.78e-14	7.96e-8
II-3([8, 4, 1], 255, [29, 31, 55])	55	1.55e-14	5.31e-7
GII([8, 2], 255, [29, 35, 55])	55	1.13e-18	5.31e-7
GII([8, 3], 255, [27, 31, 39, 59])	59	1.17e-21	3.31e-6
GII([8, 4], 255, [25, 27, 33, 41, 63])	63	9.51e-23	1.93e-5
GII([8, 5], 255, [23, 25, 29, 33, 43, 65])	65	5.06e-25	1.04e-4
GII([8, 6], 255, [21, 23, 25, 29, 35, 43, 67])	67	5.69e-26	5.15e-4
GII([8, 7], 255, [19, 21, 23, 25, 29, 35, 45, 67])	67	2.68e-26	2.34e-3

$\mathcal{C} \triangleq \{\mathbf{c} = [\mathbf{c}_0, \mathbf{c}_1, \mathbf{c}_2, \dots, \mathbf{c}_{m_1 m_2 - 1}] : \mathbf{c}_i \in \mathcal{C}_0, \forall i; \sum_{j=0}^{m_1 m_2 - 1} \alpha^{ji} \mathbf{c}_i \in \mathcal{C}_1, 1 \leq j < m_1; \sum_{i=0}^{m_1 m_2 - 1} \mathbf{c}_i \in \mathcal{C}_2\}$. This GII code allows any up to m_1 interleaves to be correctable by $\mathcal{C}_1$ (but failed by $\mathcal{C}_0$, whereas for II-3 code, it is limited to at most one interleave correctable by $\mathcal{C}_1$ (but failed by $\mathcal{C}_0$ in each stripe of $\{\mathbf{c}_{jm_2}, \mathbf{c}_{jm_2+1}, \dots, \mathbf{c}_{(j+1)m_2-1}\}$, $j = 0, 1, 2, \dots, m_1 - 1$).

Each point in Figure 4 is independently optimized on the set of nested minimum distances (associated with interleave parity lengths), using exhaustive computer search. Indeed, different points within a connected curve share the set $[m, v]$, but may have different set of nested minimum distances, as illustrated in Tables II and III. Clearly, all nested schemes noticeably outperform the simple interleaved scheme. Optimized II([8, 2]) and II([8, 3]) perform similarly, and clearly outperform other II schemes. Three-layer II scheme with middle level of 2 and 4 nodes are also investigated. They perform similarly to each other but substantially inferior to the optimized II([8, 2]). GII schemes with $v > 1$ dramatically outperform all II and II-3 schemes. The (optimized) performance of the GII scheme is monotonically superior with higher v and achieves the best at $v = m - 1 = 7$. We also observe that the gain diminishes when $v > 4$. Overall, GII([8, 4]) is appealing to trade off complexity and performance. An interesting contrast is that performance with $v = 7$ is the worst among II schemes whereas the best among

GII schemes, as shown in Figure 4. It also draws the performance of a single 12-bit RS code (where equal bit-error-rate is assumed). Interestingly, it is noticeably outperformed by the proposed GII([8, 7]) scheme when the frame-failure-rate is above $1e-12$.

Tables II and III list the nested minimum distance sets that optimize the performance of each scheme at $p_s = 0.02$ and $p_s = 0.01$, respectively. In both tables, the minimum distances of the optimized GII codes are always d_v , $v = 1, 2, \dots, 7$, furthermore, the minimum distances of GII codes are noticeably greater than those of II or II-3 code. We also observe that, all GII codes that are optimized for $p_s = 0.01$ exhibit greater minimum distances than the ones optimized for $p_s = 0.02$. This is because, the smaller the symbol-error-rate, the more minimum distance dominating performance. Noting the proposed decoder does not perform as a minimum-distance decoder. Therefore, the minimum distance in GII codes is a key performance metric but not the exclusive metric.

Both tables also list the invocation probabilities that nested decoding is invoked. Although $v = m - 1 = 7$ achieves optimal decoding failure rate, it also introduces the worst invocation rate. It is generally suggested that P_i stays below $1e-3$ in operation range, so that the average extra latency and power can be safely neglected. As can be seen from both tables, GII([8, 4]) achieves good compromise for low

decoding failure rate, low encoder/decoder complexity, and low invocation rate.

C. Erasure Correcting Algorithm for RAID Systems

This subsection specially develops an efficient erasure correcting algorithm under RAID systems (an RAID example is shown in Table I). Let a Reed-Solomon $\mathcal{C}(n, k)$ ($r \triangleq n - k$) code be considered for an RAID system, where r represents the number of parity disks and is a very small number, say $r < 10$. Note that r is not limited to even numbers for erasure decoding, differing from error-only decoding. Moreover, erasure decoding incurs no miscorrection (whereas error-only decoding incurs high miscorrection rate for small parity r).

Let $c(x)$ denote a transmitted codeword polynomial and $y(x)$ the received word polynomial where some terms are erased while the remaining terms are error-free. Let $\alpha_1, \alpha_2, \dots, \alpha_e$ ($e \leq r$) denote erasure locators, and $Y_1, Y_2, \dots, Y_e$ denote the corresponding erased magnitudes. The retrieving process starts by computing syndrome values $S_i \triangleq y(\alpha^i)$, $i = 0, 1, 2, \dots, e - 1$, where erased locations are not counted in the computation. The erased magnitudes can be solved through the following linear equation system

$$\begin{cases} Y_1 + Y_2 + \dots + Y_e = S_0 \\ Y_1\alpha_1 + Y_2\alpha_2 + \dots + Y_e\alpha_e = S_1 \\ Y_1\alpha_1^2 + Y_2\alpha_2^2 + \dots + Y_e\alpha_e^2 = S_2 \\ \vdots \\ Y_1\alpha_1^{e-1} + Y_2\alpha_2^{e-1} + \dots + Y_e\alpha_e^{e-1} = S_{e-1}. \end{cases} \quad (30)$$

Solving,

$$\begin{bmatrix} Y_1 \\ Y_2 \\ Y_3 \\ \vdots \\ Y_e \end{bmatrix} = \begin{bmatrix} 1 & 1 & \dots & 1 \\ \alpha_1 & \alpha_2 & \dots & \alpha_e \\ \alpha_1^2 & \alpha_2^2 & \dots & \alpha_e^2 \\ \vdots & \vdots & \ddots & \vdots \\ \alpha_1^{e-1} & \alpha_2^{e-1} & \dots & \alpha_e^{e-1} \end{bmatrix}^{-1} \begin{bmatrix} S_0 \\ S_1 \\ S_2 \\ \vdots \\ S_{e-1} \end{bmatrix}. \quad (31)$$

Given $e \leq r$ erasures, it suffices to compute the first e syndromes, $S_0, S_1, \dots, S_{e-1}$, although there are r valid syndromes. On the other hand, when $e > r$, the erasures are not retrievable since there exist many valid solutions.

It is straightforward to import the preceding error-only decoding algorithm to erasure-only decoding for GII-RS codes. Herein we reveal several specialties to further reduce complexity. In an RAID system, an enormous set of erased values (from several disks) need to be retrieved given the same erasure locations. Therefore, matrix inversion in (31) is pre-computed and results in negligible computational cost. Since erasure counts are known *a priori*, the decoding failure can be pre-determined without going through decoding procedure, likewise, when the number of tentatively generated syndromes is smaller than the given number of erasures, performing erasure correction is fruitless. When the erasures are correctable, i.e., the erasure count is no greater than parity count, it suffices to generate the erasure count of syndromes. In addition, recursive LFSR (21) is not readily applicable for computing higher order syndromes, as the erasure locator polynomial is not handy, therefore, it becomes more efficient to directly

compute higher order syndromes. We are now ready to present our new decoding algorithm.

Erasure Correcting Algorithm for GII-RS Codes

Input: $y(x) = [y_0(x), y_1(x), \dots, y_{m-1}(x)]$, $\{\alpha_j^{(0)}\}_{j=1}^{e_0}$, $\{\alpha_j^{(1)}\}_{j=1}^{e_1}$, $\dots$, $\{\alpha_j^{(m-1)}\}_{j=1}^{e_{m-1}}$.

- 1) Reorder the erasure count sequence $e_0, e_1, \dots, e_{m-1}$, to the new sequence $\tau_0, \tau_1, \dots, \tau_{m-1}$, following (24). If there exists $\tau_i > r_i$, $0 \leq i \leq v$, then declare failure.
- 2) Compute syndromes $\{S_j^{(i)}\}_{j=0}^{r-1}$, $i = 0, 1, \dots, m-1$, where $r = e_i$ if $e_i \leq r_0$, otherwise $r = r_0$.
- 3) Correct erasures through (31) for interleaves that have up to r_0 erasures.
- 4) If all successful, then return the corrected codeword $c(x) = [c_0(x), c_1(x), \dots, c_{m-1}(x)]$; else let the uncorrectable interleave indexes be $I = \{l_1, l_2, \dots, l_b\}$ ($b \leftarrow |I|$).
- 5) Compute corrupted nested interleaves

$$\tilde{y}_i(x) = \sum_{j \in I} \alpha_j^{ij} y_j(x) + \sum_{j \in I^c} \alpha_j^{ij} c_j(x) \quad (32)$$

for $i = 0, 1, 2, \dots, b-1$.

- 6) Compute higher order syndromes $\{\tilde{S}_j^{(i)}\}_{j=\tau_0}^{\tau_{b-i}-1}$ over each nested interleaves $\tilde{y}_i(x)$, $i = 0, 1, \dots, b-1$ (Note the number of computed syndromes are dependent on erasure counts).
- 7) While $b > 0$, do:
 - a) Compute syndromes $\{S_j^{(li)}\}_{j=\tau_{v-b'}}^{\tau_{v-b}-1}$, $i = 1, 2, \dots, b$, through (33) as shown at the top of the next page, where $\tau_{v-b'}$ is initialized to r_0 .
 - b) Correct erasures through (31) for interleaves that have τ_{v-b} erasures.
 - c) Let I' be the index set of newly corrected interleaves. Set $I \leftarrow I - I'$, $b' \leftarrow b$, and $b \leftarrow |I|$. If $I = \emptyset$ (all remaining interleaves are corrected) then return the corrected codeword.
 - d) Update higher order nested syndromes through

$$\left\{ \tilde{S}_j^{(l)} \leftarrow \tilde{S}_j^{(l)} - \sum_{i \in I'} \alpha^{il} \sum_{z=1}^{e_i} Y_z^{(i)} (\alpha_z^{(i)})^j \right\}_{j=\tau_{v-b'}}$$

for $0 \leq l < b$.

Note in RAID recovery, the inverted matrix in (33) is used many times for rebuilding entire disks, therefore, it is more efficient to pre-compute it.

It is instrumental to compare the above algorithm against the erasure correcting algorithm given in [9] and [10]. There lie three notable differences. (i). The algorithm in [9] and [10] requires to dynamically transform the parity check matrix to an upper triangular shape (e.g., using Gaussian elimination), while the above does not. (ii). The above requires to compute the corrupted nested interleaves and compute higher order syndromes of uncorrectable interleaves through de-mapping in (33), while the algorithm in [9] and [10] does not. (iii). The above dynamically reduces erasure correcting capabilities to the actual erasure counts (to dynamically cut complexity), while the algorithm in [9] and [10] does not.

$$\begin{bmatrix} S_j^{(l_1)} \\ S_j^{(l_2)} \\ \vdots \\ S_j^{(l_b)} \end{bmatrix}_{j=\tau_{v-b'}}^{\tau_{v-b}-1} = \begin{bmatrix} 1 & 1 & \dots & 1 \\ \alpha^{l_1} & \alpha^{l_2} & \dots & \alpha^{l_b} \\ \vdots & \vdots & \ddots & \vdots \\ \alpha^{(b-1)l_1} & \alpha^{(b-1)l_2} & \dots & \alpha^{(b-1)l_b} \end{bmatrix}^{-1} \begin{bmatrix} \tilde{S}_j^{(0)} \\ \tilde{S}_j^{(1)} \\ \vdots \\ \tilde{S}_j^{(b-1)} \end{bmatrix}_{j=\tau_{v-b'}}^{\tau_{v-b}-1} \quad (33)$$

The following theorem characterizes the above erasure correction algorithm. Its proof trivially follows (31) and the proof of Theorem 2.

Theorem 3: Let $e_0, e_1, \dots, e_{m-1}$ denote the number of erasures over received interleaves $y_0(x), y_1(x), \dots, y_{m-1}(x)$, respectively. Let $\tau_0, \tau_1, \dots, \tau_{m-1}$ be reordered erasure counts following (24). Then, the decoding is successful iff

$$\tau_i \leq r_i, \quad i = 0, 1, 2, \dots, v. \quad (34)$$

D. Systematic Encoding Algorithm

All cyclic codes, including RS and BCH codes, can be systematically encoded by a simple linear-feedback-shift-register (LFSR) circuit [16, pp. 235–237]. Specifically, an LFSR encoder systematically encodes a data polynomial $a(x)$ to a systematic codeword $c(x)$ through iteratively dividing $-x^r a(x)$ by a generator polynomial $g(x)$. A systematic codeword is comprised of the original data polynomial $a(x)$ followed by its resulting remainder polynomial $p(x)$ [16, pp. 235–237]. Herein we denote the LFSR encoding by

$$\mathcal{E}(a(x), g(x)) \rightarrow c(x) \triangleq [a(x), p(x)]. \quad (35)$$

It is worth emphasizing that a systematic codeword is reversely indexed in polynomial representation, i.e., $\mathbf{c} = [c_{n-1}, c_{n-2}, \dots, c_1, c_0]$. In this section we present an efficient systematic encoding algorithm utilizing LFSR.

To ease presentation, we introduce two operators. $\mathcal{L}_r$ truncates a polynomial $a(x)$ to keep only its lowest r terms,

$$\begin{aligned} \mathcal{L}_r a(x) &\triangleq a(x) \bmod x^r \quad \text{or} \\ \mathcal{L}_r [a_{l-1}, a_{l-2}, \dots, a_0] &\triangleq [a_{r-1}, a_{r-2}, \dots, a_0]. \end{aligned} \quad (36)$$

$\mathcal{U}_r$ truncates a polynomial $a(x)$ to keep its upper terms starting with power x^r .

$$\begin{aligned} \mathcal{U}_r a(x) &\triangleq (a(x) - \mathcal{L}_r a(x)) / x^r \quad \text{or} \\ \mathcal{U}_r [a_{l-1}, a_{l-2}, \dots, a_0] &\triangleq [a_{l-1}, a_{l-2}, \dots, a_r]. \end{aligned} \quad (37)$$

We first explore systematic encoding for the special case $v = 1$. Let $g_0(x)$ and $g_1(x)$ be the generator polynomials of $\mathcal{C}_0$ and $\mathcal{C}_1$ respectively. Let $a_0(x), a_1(x), a_2(x), \dots, a_{m-1}(x)$ be data polynomials, satisfying $\deg(a_0(x)) < k_1$ and $\deg(a_i(x)) < k_0, 1 \leq i < m$.

Firstly apply LFSR encoding to $a_1(x), a_2(x), \dots, a_{m-1}(x)$, respectively, with respect to $g_0(x)$,

$$\mathcal{E}(a_i(x), g_0(x)) \rightarrow c_i(x) \triangleq [a_i(x), p_i(x)] \quad (38)$$

for $i = 1, 2, \dots, m-1$. Secondly apply LFSR encoding to the left-aligned summation $a_0(x) + \sum_{i=1}^{m-1} \mathcal{U}_{r_1-r_0} a_i(x)$ with

respect to $g_1(x)$

$$\begin{aligned} &\mathcal{E}\left(a_0(x) + \sum_{i=1}^{m-1} \mathcal{U}_{r_1-r_0} a_i(x), g_1(x)\right) \\ &\rightarrow c_0^*(x) \triangleq \left[a_0(x) + \sum_{i=1}^{m-1} \mathcal{U}_{r_1-r_0} a_i(x), p_0^*(x) \right]. \end{aligned} \quad (39)$$

Then, the following $c_0(x)$ is the desired codeword to encode $a_0(x)$,

$$\begin{aligned} c_0(x) &\triangleq c_0^*(x) - \sum_{i=1}^{m-1} c_i(x) \\ &= \left[a_0(x), p_0^*(x) - \sum_{i=1}^{m-1} \mathcal{L}_{r_1} c_i(x) \right]. \end{aligned} \quad (40)$$

It can be easily verified that $c_0(x) \in \mathcal{C}_0$, and furthermore, $c_0(x) + \sum_{i=1}^{m-1} c_i(x) = c_0^*(x)$ is a codeword polynomial in $\mathcal{C}_1$. Figure 5 depicts the block diagram of this encoding method for the case $m = 4, v = 1$, where the notation “Clk” denotes the clock number.

We next extend the above method to the general scenario $v > 1$. Let $a_0(x), a_1(x), a_2(x), \dots, a_{m-1}(x)$ be data polynomials, satisfying $\deg(a_i(x)) < k_{v-i}, 0 \leq i < v$, and $\deg(a_j(x)) < k_0, v \leq j < m$. Assume that $a_v(x), a_{v+1}(x), \dots, a_{m-1}(x)$, are systematically encoded to $c_v(x), c_{v+1}(x), \dots, c_{m-1}(x)$, respectively, utilizing the generator polynomial $g_0(x)$. We aim to determine systematic codewords, $c_i(x) \triangleq [a_i(x), p_i(x)]$ ($\deg(p_i(x)) < r_{v-i}, i = 0, 1, \dots, v-1$), satisfying (41), as shown at the bottom of the next page.

It is necessary to validate (41), by showing that its solution must satisfy $c_0(x), c_1(x), \dots, c_{v-1}(x) \in \mathcal{C}_0$. Recall that $c_i(x), i = v, v+1, \dots, m-1$, all divide $g_0(x)$, thus, (41) can be interpreted as (42), as shown at the bottom of the next page, which is equivalent to

$$\begin{bmatrix} 1 & 1 & \dots & 1 \\ 1 & \alpha & \dots & \alpha^{v-1} \\ 1 & \alpha^2 & \dots & \alpha^{2(v-1)} \\ \vdots & \vdots & \ddots & \vdots \\ 1 & \alpha^{v-1} & \dots & \alpha^{(v-1)^2} \end{bmatrix} \cdot \begin{bmatrix} c_0(x) \bmod g_0(x) \\ c_1(x) \bmod g_0(x) \\ c_2(x) \bmod g_0(x) \\ \vdots \\ c_{v-1}(x) \bmod g_0(x) \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \\ \vdots \\ 0 \end{bmatrix}.$$

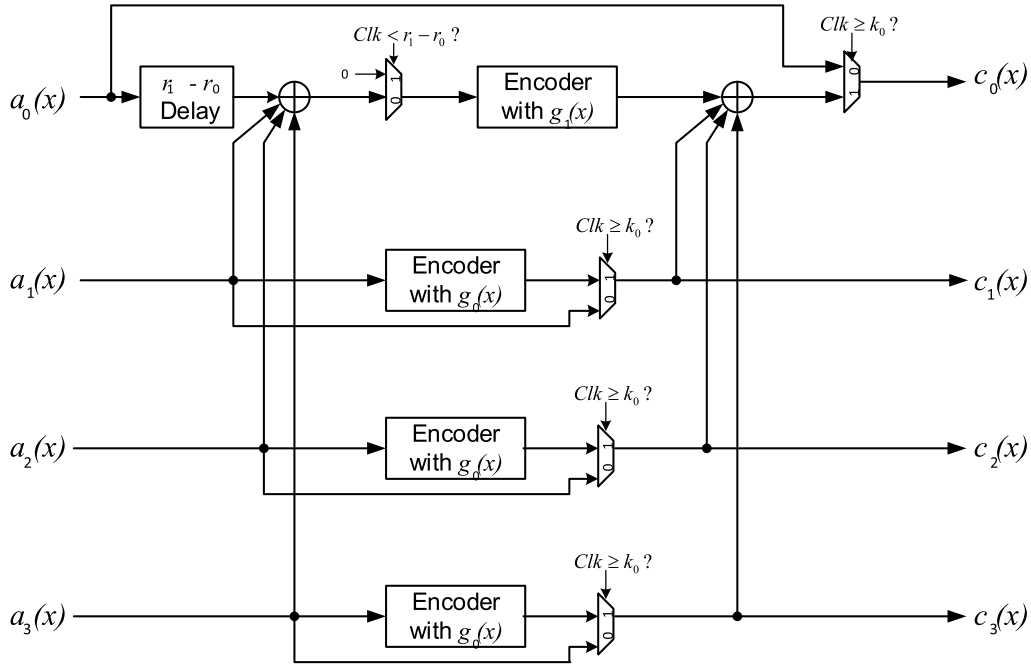


Fig. 5. The encoder block diagram of the integrated interleaved code with $m = 4$, $v = 1$.

Note the above matrix is Vandermonde and thus is non-singular. Therefore,

$$c_j(x) \equiv 0 \pmod{g_0(x)}, \quad j = 0, 1, 2, \dots, v-1, \quad (43)$$

which concludes that (41) is a sufficient system to determine the parity polynomials $p_0(x)$, $p_1(x)$, ..., $p_{v-1}(x)$.

It is instrumental to first solve for II-RS code where $r_1 = r_2 = \dots = r_v$, which effectively reduces (41) to (44), as shown at the bottom of this page.

Solving, we obtain

$$\begin{bmatrix} c_0(x) \pmod{g_1(x)} \\ c_1(x) \pmod{g_1(x)} \\ \vdots \\ c_{v-1}(x) \pmod{g_1(x)} \end{bmatrix} = \Pi \cdot \begin{bmatrix} c_v(x) \pmod{g_1(x)} \\ c_{v+1}(x) \pmod{g_1(x)} \\ \vdots \\ c_{m-1}(x) \pmod{g_1(x)} \end{bmatrix} \quad (45)$$

$$\begin{cases} c_0(x) + c_1(x) + \dots + c_{v-1}(x) \equiv -\sum_{i=v}^{m-1} c_i(x) \pmod{g_v(x)} \\ c_0(x) + \alpha c_1(x) + \dots + \alpha^{v-1} c_{v-1}(x) \equiv -\sum_{i=v}^{m-1} \alpha^i c_i(x) \pmod{g_{v-1}(x)} \\ c_0(x) + \alpha^2 c_1(x) + \dots + \alpha^{2(v-1)} c_{v-1}(x) \equiv -\sum_{i=v}^{m-1} \alpha^{2i} c_i(x) \pmod{g_{v-2}(x)} \\ \vdots \\ c_0(x) + \alpha^{v-1} c_1(x) + \dots + \alpha^{(v-1)(v-1)} c_{v-1}(x) \equiv -\sum_{i=v}^{m-1} \alpha^{i(v-1)} c_i(x) \pmod{g_1(x)} \end{cases} \quad (41)$$

$$\begin{cases} c_0(x) + c_1(x) + \dots + c_{v-1}(x) \equiv 0 \pmod{g_0(x)} \\ c_0(x) + \alpha c_1(x) + \dots + \alpha^{v-1} c_{v-1}(x) \equiv 0 \pmod{g_0(x)} \\ c_0(x) + \alpha^2 c_1(x) + \dots + \alpha^{2(v-1)} c_{v-1}(x) \equiv 0 \pmod{g_0(x)} \\ \vdots \\ c_0(x) + \alpha^{v-1} c_1(x) + \dots + \alpha^{(v-1)^2} c_{v-1}(x) \equiv 0 \pmod{g_0(x)} \end{cases} \quad (42)$$

$$\begin{cases} c_0(x) + c_1(x) + \dots + c_{v-1}(x) \equiv -\sum_{i=v}^{m-1} c_i(x) \pmod{g_1(x)} \\ c_0(x) + \alpha c_1(x) + \dots + \alpha^{v-1} c_{v-1}(x) \equiv -\sum_{i=v}^{m-1} \alpha^i c_i(x) \pmod{g_1(x)} \\ c_0(x) + \alpha^2 c_1(x) + \dots + \alpha^{2(v-1)} c_{v-1}(x) \equiv -\sum_{i=v}^{m-1} \alpha^{2i} c_i(x) \pmod{g_1(x)} \\ \vdots \\ c_0(x) + \alpha^{v-1} c_1(x) + \dots + \alpha^{(v-1)(v-1)} c_{v-1}(x) \equiv -\sum_{i=v}^{m-1} \alpha^{i(v-1)} c_i(x) \pmod{g_1(x)} \end{cases} \quad (44)$$

$$\mathbf{\Pi} \triangleq - \begin{bmatrix} 1 & 1 & \dots & 1 \\ 1 & \alpha & \dots & \alpha^{v-1} \\ 1 & \alpha^2 & \dots & \alpha^{2(v-1)} \\ \vdots & \vdots & \ddots & \vdots \\ 1 & \alpha^{v-1} & \dots & \alpha^{(v-1)^2} \end{bmatrix}^{-1} \cdot \begin{bmatrix} 1 & 1 & \dots & 1 \\ \alpha^v & \alpha^{v+1} & \dots & \alpha^{m-1} \\ \alpha^{2v} & \alpha^{2(v+1)} & \dots & \alpha^{2(m-1)} \\ \vdots & \vdots & \ddots & \vdots \\ \alpha^{(v-1)v} & \alpha^{(v-1)(v+1)} & \dots & \alpha^{(v-1)(m-1)} \end{bmatrix} \quad (46)$$

where the coefficient matrix $\mathbf{\Pi}$ is pre-computed through (46), as shown at the top of this page. Denote by $p_i^*(x)$ the parity polynomial of the message polynomial $a_i(x)$ with respect to $g_1(x)$, i.e.,

$$\mathcal{E}(a_i(x), g_1(x)) \rightarrow [a_i(x), p_i^*(x)] \quad (47)$$

for $i = 0, 1, \dots, v-1$, and $p'_j(x)$ the parity polynomial of the truncated message polynomial $\mathcal{U}_{r_1-r_0}a_j(x)$, i.e.,

$$\mathcal{E}(\mathcal{U}_{r_1-r_0}a_j(x), g_1(x)) \rightarrow [\mathcal{U}_{r_1-r_0}a_j(x), p'_j(x)] \quad (48)$$

$j = v, v+1, \dots, m-1$. Then, we obtain

$$\begin{aligned} c_i(x) \bmod g_1(x) &= p_i(x) - p_i^*(x), \quad 0 \leq i < v, \\ c_j(x) \bmod g_1(x) &= \mathcal{L}_{r_1}c_j(x) - p'_j(x), \quad v \leq j < m, \end{aligned} \quad (49)$$

which leads to re-express (45)

$$\begin{bmatrix} p_0(x) - p_0^*(x) \\ p_1(x) - p_1^*(x) \\ \vdots \\ p_{v-1}(x) - p_{v-1}^*(x) \end{bmatrix} = \mathbf{\Pi} \cdot \begin{bmatrix} \mathcal{L}_{r_1}c_v(x) - p'_v(x) \\ \mathcal{L}_{r_1}c_{v+1}(x) - p'_{v+1}(x) \\ \vdots \\ \mathcal{L}_{r_1}c_{m-1}(x) - p'_{m-1}(x) \end{bmatrix}.$$

Consequently, we obtain the desired parity polynomials $p_0(x), p_1(x), \dots, p_{v-1}(x)$ for II-RS encoding

$$\begin{bmatrix} p_0(x) \\ p_1(x) \\ \vdots \\ p_{v-1}(x) \end{bmatrix} = \begin{bmatrix} p_0^*(x) \\ p_1^*(x) \\ \vdots \\ p_{v-1}^*(x) \end{bmatrix} - \mathbf{\Pi} \cdot \begin{bmatrix} p'_v(x) - \mathcal{L}_{r_1}c_v(x) \\ p'_{v+1}(x) - \mathcal{L}_{r_1}c_{v+1}(x) \\ \vdots \\ p'_{m-1}(x) - \mathcal{L}_{r_1}c_{m-1}(x) \end{bmatrix}. \quad (50)$$

We now explore the general case $r_1 \leq r_2 \leq \dots \leq r_v$. Observe that the linear system (44) suffices to determine the parity polynomial $p_{v-1}(x)$. Denote by $\mathbf{\Gamma}^{(i)}$ the inverse matrix

$$\begin{aligned} \mathbf{\Gamma}^{(i)} &\triangleq [\Gamma_0^{(i)}, \Gamma_1^{(i)}, \dots, \Gamma_i^{(i)}]^T \\ &\triangleq \begin{bmatrix} 1 & 1 & 1 & \dots & 1 \\ 1 & \alpha & \alpha^2 & \dots & \alpha^i \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & \alpha^i & \alpha^{2i} & \dots & \alpha^{i^2} \end{bmatrix}^{-1} \end{aligned} \quad (51)$$

and by $\mathbf{\Theta}^{(i)}$ the matrix

$$\mathbf{\Theta}^{(i)} \triangleq - \begin{bmatrix} 1 & 1 & \dots & 1 \\ \alpha^{i+1} & \alpha^{i+2} & \dots & \alpha^{m-1} \\ \vdots & \vdots & \ddots & \vdots \\ \alpha^{(i+1)i} & \alpha^{(i+2)i} & \dots & \alpha^{(m-1)i} \end{bmatrix} \quad (52)$$

for $i = 0, 1, 2, \dots, v-1$. By pre-computing the coefficient vector

$$\boldsymbol{\pi}^{(i)} \triangleq \mathbf{\Gamma}_i^{(i)} \cdot \mathbf{\Theta}^{(i)}, \quad (53)$$

$c_{v-1}(x)$ is explicitly expressed

$$c_{v-1}(x) \equiv \boldsymbol{\pi}^{(v-1)} \cdot [c_v(x), c_{v+1}(x), \dots, c_{m-1}(x)]^T \pmod{g_1(x)}. \quad (54)$$

Following (49), the desired parity polynomial $p_{v-1}(x)$ is explicitly determined by (55), as shown at the top of the next page.

Upon solving $c_{v-1}(x)$, we then determine $c_{v-2}(x)$ through the equation system (56), as shown at the top of the next page deducing from (41). Subsequently, we obtain

$$c_{v-2}(x) \equiv \boldsymbol{\pi}^{(v-2)} \cdot [c_{v-1}(x), c_v(x), \dots, c_{m-1}(x)]^T \pmod{g_2(x)} \quad (57)$$

and go on by induction to conclude with the following theorem.

Theorem 4: Given a set of message polynomials $a_i(x)$ ($\deg(a_i(x)) < k_{v-i}$), $i = 0, 1, \dots, v-1$, $a_j(x)$ ($\deg(a_j(x)) < k_0$), $j = v, v+1, \dots, m-1$, its systematic encoding toward a codeword polynomial $[c_0(x), c_1(x), \dots, c_{m-1}(x)]$ in GII-RS($[m, v], n$) defined in (3) is achieved by

- (i). generating $c_j(x)$ by applying LFSR encoding to $a_j(x)$ with respect to the first-layer generator polynomial $g_0(x)$, for $j = v, v+1, \dots, m-1$;
- (ii). sequentially determining $c_{v-1}(x), c_{v-2}(x), \dots, c_1(x), c_0(x)$, in the form of

$$c_i(x) \equiv \boldsymbol{\pi}^{(i)} \cdot [c_{i+1}(x), c_{i+2}(x), \dots, c_{m-1}(x)]^T \pmod{g_{v-i}(x)} \quad (58)$$

for $i = v-1, v-2, \dots, 1, 0$.

We are now ready to describe the detailed encoding algorithm.

II-RS Encoding Algorithm

- 1) Apply LFSR encoding to $a_v(x), a_{v+1}(x), \dots, a_{m-1}(x)$, respectively, with respect to $g_0(x)$.

$$\mathcal{E}(a_i(x), g_0(x)) \rightarrow c_i(x) \triangleq [a_i(x), p_i(x)]$$

for $i = v, v+1, \dots, m-1$, where $\deg(p_i(x)) < r_0$, $v \leq i < m$.

- 2) For $i = v-1, v-2, \dots, 1, 0$, do:

- a) Compute message $a_i^*(x)$

$$a_i^*(x) = a_i(x) + \boldsymbol{\pi}^{(i)} \cdot [\mathcal{U}_{r_v-i-r_0}[a_{i+1}(x), a_{i+2}(x), \dots, a_{m-1}(x)]]^T$$

where $\boldsymbol{\pi}^{(i)}$ is defined in (53).

$$p_{v-1}(x) = p_{v-1}^*(x) - \pi^{(v-1)} \cdot [\mathcal{L}_{r_1} c_v(x) - p'_v(x), \mathcal{L}_{r_1} c_{v+1}(x) - p'_{v+1}(x), \dots, \mathcal{L}_{r_1} c_{m-1}(x) - p'_{m-1}(x)]^T \quad (55)$$

$$\begin{cases} c_0(x) + c_1(x) + \dots + c_{v-2}(x) \equiv -\sum_{i=v-1}^{m-1} c_i(x) \pmod{g_2(x)} \\ c_0(x) + \alpha c_1(x) + \dots + \alpha^{(v-2)} c_{v-2}(x) \equiv -\sum_{i=v-1}^{m-1} \alpha^i c_i(x) \pmod{g_2(x)} \\ \vdots \\ c_0(x) + \alpha^{v-2} c_1(x) + \dots + \alpha^{(v-2)^2} c_{v-2}(x) \equiv -\sum_{i=v-1}^{m-1} \alpha^{i(v-2)} c_i(x) \pmod{g_2(x)}. \end{cases} \quad (56)$$

- b) Apply LFSR encoding to $a_i^*(x)$ with respect to $g_{v-i}(x)$

$$\mathcal{E}(a_i^*(x), g_{v-i}(x)) \rightarrow [a_i^*(x), p_i^*(x)].$$

- c) Determine the parity polynomial $p_i(x)$ associated with $a_i(x)$

$$p_i(x) = p_i^*(x) - \pi^{(i)} \cdot \mathcal{L}_{r_{v-i}}[c_{i+1}(x), c_{i+2}(x), \dots, c_{m-1}(x)]^T.$$

It is interesting to measure the above computational complexity. LFSR encoding in Steps 1 and 2.(b) together takes $O(Rmn^2)$ finite field operations, where $R = \sum_{i=0}^{m-1} r_i/nm$ denotes the code rate. Steps 2.(a) and 2.(c) together take $O(vmn)$ finite field operations. Summing them up, the algorithmic complexity is $O(mn(v + Rn))$.

Note in the above algorithm, $\pi^{(0)}$ is an all-one vector, whereas $\pi^{(0)} = [\alpha^v, \alpha^{2v}, \dots, \alpha^{(m-1)v}]$ if following the original definition in [8].

Finally, we remark that, although equal length among interleaves is assumed throughout the context, this restriction can be relaxed without affecting the proposed encoding and decoding algorithms. This is particularly useful for applications where equal data (but not code) length must be enforced among all interleaves.

III. GENERALIZED INTEGRATED INTERLEAVED BCH CODES

We note that the preceding II/GII concept is only applicable for binary Bose-Chaudhuri-Hocquenghem (BCH) codes for the simplest case $v = 1$. This is because, the integrating coefficients α^{bi} given in (3) are only valid with $b = 0$. In this section, we present a novel GII scheme over BCH codes. The main idea lies in the following associative binary polynomial defined on each $\gamma \in \text{GF}(2^q)$,

$$\gamma(x) \triangleq \gamma_{q-1}x^{q-1} + \gamma_{q-2}x^{q-2} + \dots + \gamma_1x + \gamma_0 \quad (59)$$

where $\gamma_i \in \text{GF}(2)$, $i = 0, 1, \dots, q-1$, and

$$\begin{aligned} \gamma &\triangleq [\gamma_{q-1}, \gamma_{q-2}, \dots, \gamma_1, \gamma_0] \\ &= \gamma_{q-1}\alpha^{q-1} + \gamma_{q-2}\alpha^{q-2} + \dots + \gamma_1\alpha + \gamma_0. \end{aligned}$$

The function (59) carries the following desirable properties (readers are referred to [16] for their proofs),

$$\alpha(x^j) = x^j \pmod{\psi(x)} \quad (60)$$

where α denotes a primitive element in $\text{GF}(2^q)$ that is defined by a primitive polynomial $\psi(x)$.

$$\alpha(\gamma) = \gamma, \quad \forall \gamma \in \text{GF}(2^q), \quad (61)$$

$$\alpha^i(x) = \alpha(x^i), \quad 0 \leq i < 2^q, \quad (62)$$

$$\alpha^{-1}(x) = \alpha(x)^{-1} \pmod{\psi(x)}. \quad (63)$$

A t -correcting BCH code $\mathcal{C}(n, k)$ defined over the Galois field $\text{GF}(2^q)$ has the following generator polynomial

$$g(x) \triangleq \text{LCM}(m_1(x), m_3(x), \dots, m_{2t-1}(x)) \quad (64)$$

where LCM denotes “the least common multiple”, and $m_i(x)$ denotes the minimal (binary) polynomial to contain the root α^i .

Consider a relaxed subcode definition such that $\mathcal{C}_1(n_1, k_1) \subset \mathcal{C}_0(n_0, k_0)$ if their generator polynomials satisfy $g_0(x) \mid g_1(x)$ (while allowing different code length). Let $\{\mathcal{C}_i(n_i, k_i, d_i)\}_{i=0}^v$ be binary BCH codes satisfying

$$\begin{aligned} \mathcal{C}_{i_s} &= \dots = \mathcal{C}_{i_{s-1}+1} \subset \mathcal{C}_{i_{s-1}} = \dots = \mathcal{C}_{i_{s-2}+1} \\ &\subset \dots \subset \mathcal{C}_{i_1+1} = \dots = \mathcal{C}_1 \subset \mathcal{C}_0 \end{aligned} \quad (65)$$

where $i_0 = 0$ and $i_s = v$. A GII-BCH code is given by

$$\mathcal{C} \triangleq \left\{ c(x) = [c_0(x), c_1(x), \dots, c_{m-1}(x)] : c_i(x) \in \mathcal{C}_0, \forall i, \sum_{i=0}^{m-1} \alpha(x^{bi})c_i(x) \in \mathcal{C}_{v-b}, 0 \leq b < v \right\}, \quad (66)$$

where $v < m < 2^q$. The next lemma is critical to assert the basic properties of GII-BCH codes.

Lemma 2: Let $c_i(x)$, $i = 0, 1, \dots, v-1$, be BCH codewords in $\mathcal{C}_0$. If (67) lies in $\mathcal{C}_1$, where $\mathcal{C}_1 \subset \mathcal{C}_0$, then $c_i(x)$, $i = 0, 1, \dots, v-1$, must lie in $\mathcal{C}_1$.

Proof: It suffices to show that each $c_i(x)$, $i = 0, 1, \dots, v-1$, contains all roots of $\frac{g_1(x)}{g_0(x)}$. Let γ be a root of $\frac{g_1(x)}{g_0(x)}$. Evaluating (67), as shown at the top of the next page with $x = \gamma$, we obtain

$$\begin{bmatrix} 1 & 1 & \dots & 1 \\ 1 & \gamma & \dots & \gamma^{v-1} \\ \vdots & \vdots & \ddots & \vdots \\ 1 & \gamma^{v-1} & \dots & \gamma^{(v-1)(v-1)} \end{bmatrix} \begin{bmatrix} c_0(\gamma) \\ c_1(\gamma) \\ \vdots \\ c_{v-1}(\gamma) \end{bmatrix} = 1 \begin{bmatrix} 0 \\ 0 \\ \vdots \\ 0 \end{bmatrix}.$$

Since the connection matrix is non-singular, the above equality indicates

$$c_i(\gamma) = 0, \quad i = 0, 1, \dots, v-1.$$

Therefore, $c_i(x)$, $i = 0, 1, \dots, v-1$, all divide $g_1(x)$. $\square\square$

$$\begin{bmatrix} c'_0(x) \\ c'_1(x) \\ \vdots \\ c'_{v-1}(x) \end{bmatrix} \triangleq \begin{bmatrix} 1 & 1 & \dots & 1 \\ 1 & \alpha(x) & \dots & \alpha(x^{v-1}) \\ \vdots & \vdots & \ddots & \vdots \\ 1 & \alpha(x^{v-1}) & \dots & \alpha(x^{(v-1)(v-1)}) \end{bmatrix} \begin{bmatrix} c_0(x) \\ c_1(x) \\ \vdots \\ c_{v-1}(x) \end{bmatrix} \quad (67)$$

Theorem 5: Let d_i^* , $i = 0, 1, \dots, v$, denote the actual (against the designed) minimum distance of the BCH code C_i . The GII-BCH code $\mathcal{C}(N, K, d_{\min}^*)$ is a binary linear code of length $N = mn$, dimension $K = \sum_{i=1}^v k_i + (m-v)k_0$, and minimum distance bounded by

$$d_{\min}^* \geq \min \{ (v+1)d_0^*, (v-i_1+1)d_1^*, \dots, (v-i_{s-1}+1)d_{i_{s-1}}^*, d_v^* \}. \quad (68)$$

Proof: It follows from the proof of Theorem 1 and Lemma 2. In particular, if there are i ($i \leq v$) non-zero interleaves, then they must all lie in C_{v-i+1} by Lemma 2. This concludes the lower bound of minimum distance. $\square\square$

We note that unlike a GII-RS code whose minimum distance is precisely determined, the minimum distance of a GII-BCH code is loosely bounded. Following the literature of BCH characterizations, we define the designed minimum distance, denoted by $d_{\min}$, for a GII-BCH code as

$$d_{\min} \triangleq \min \{ (v+1)d_0^*, (v-i_1+1)d_1^*, \dots, (v-i_{s-1}+1)d_{i_{s-1}}^*, d_v^* \}. \quad (69)$$

A. Systematic Encoding Algorithm

We proceed to develop a systematic encoding algorithm which is modified from the preceding encoding algorithm for GII-RS codes. Let $a_0(x), a_1(x), a_2(x), \dots, a_{m-1}(x)$ be data polynomials, satisfying $\deg(a_i(x)) < k_{v-i}$, $0 \leq i < v$, and $\deg(a_j(x)) < k_0$, $v \leq j < m$. Assume that $a_v(x), a_{v+1}(x), \dots, a_{m-1}(x)$, are systematically encoded to $c_v(x), c_{v+1}(x), \dots, c_{m-1}(x)$, respectively, utilizing the generator polynomial $g_0(x)$. By Lemma 2, it is sufficient to determine systematic codewords, $c_i(x) \triangleq [a_i(x), p_i(x)]$ ($\deg(p_i(x)) < r_{v-i}$), $i = 0, 1, \dots, v-1$, satisfying (70), as shown at the bottom of this page.

Note that the above system operates in GF(2), wherein “−” operation is equivalent to “+” operation.

To solve for $c_{v-1}(x)$, we again relax from $\text{mod } g_i(x)$, $i = 1, 2, \dots, v$, in the above system to $\text{mod } g_1(x)$. Denote by

$\Gamma^{(i)}(x)$, $0 \leq i < v$, the inverse matrix

$$\begin{aligned} \Gamma^{(i)}(x) &\triangleq [\Gamma_0^{(i)}(x), \Gamma_1^{(i)}(x), \dots, \Gamma_i^{(i)}(x)]^T \\ &\triangleq \begin{bmatrix} 1 & 1 & 1 & \dots & 1 \\ 1 & \alpha(x) & \alpha^2(x) & \dots & \alpha^i(x) \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & \alpha^i(x) & \alpha^{2i}(x) & \dots & \alpha^{i^2}(x) \end{bmatrix}^{-1} \pmod{g_{v-i}(x)}, \end{aligned} \quad (71)$$

whose well-definedness is asserted by Lemma 2. Define

$$\Theta^{(i)}(x) \triangleq \begin{bmatrix} 1 & 1 & \dots & 1 \\ \alpha^{i+1}(x) & \alpha^{i+2}(x) & \dots & \alpha^{m-1}(x) \\ \vdots & \vdots & \ddots & \vdots \\ \alpha^{(i+1)i}(x) & \alpha^{(i+2)i}(x) & \dots & \alpha^{(m-1)i}(x) \end{bmatrix}, \quad (72)$$

for $0 \leq i < v$. By pre-computing the coefficient vector

$$\pi^{(i)}(x) \triangleq \Gamma_i^{(i)}(x) \cdot \Theta^{(i)}(x) \pmod{g_{v-i}(x)}, \quad (73)$$

$c_{v-1}(x)$ is explicitly expressed

$$\begin{aligned} c_{v-1}(x) &\equiv \pi^{(v-1)}(x) \cdot [c_v(x), c_{v+1}(x), \\ &\dots, c_{m-1}(x)]^T \pmod{g_1(x)}. \end{aligned} \quad (74)$$

And so forth, $c_{v-2}(x), c_{v-3}(x), \dots, c_1(x), c_0(x)$, are computed sequentially

$$\begin{aligned} c_i(x) &\equiv \pi^{(i)}(x) \cdot [c_{i+1}(x), c_{i+2}(x), \\ &\dots, c_{m-1}(x)]^T \pmod{g_{v-i}(x)}, \end{aligned} \quad (75)$$

for $i = v-1, v-2, \dots, 1, 0$. The detailed encoding procedure is described below.

GII-BCH Encoding Algorithm

- 1) Apply LFSR encoding to $a_v(x), a_{v+1}(x), \dots, a_{m-1}(x)$, respectively, with respect to $g_0(x)$

$$\mathcal{E}(a_i(x), g_0(x)) \rightarrow c_i(x) \triangleq [a_i(x), p_i(x)]$$

for $i = v, v+1, \dots, m-1$, where $\deg(p_i(x)) < r_0$, $v \leq i < m$.

$$\begin{cases} c_0(x) + c_1(x) + \dots + c_{v-1}(x) & \equiv \sum_{i=v}^{m-1} c_i(x) \pmod{g_v(x)} \\ c_0(x) + \alpha(x)c_1(x) + \dots + \alpha(x^{v-1})c_{v-1}(x) & \equiv \sum_{i=v}^{m-1} \alpha(x^i)c_i(x) \pmod{g_{v-1}(x)} \\ c_0(x) + \alpha(x^2)c_1(x) + \dots + \alpha(x^{2(v-1)})c_{v-1}(x) & \equiv \sum_{i=v}^{m-1} \alpha(x^{2i})c_i(x) \pmod{g_{v-2}(x)} \\ \vdots & \vdots \\ c_0(x) + \alpha(x^{v-1})c_1(x) + \dots + \alpha(x^{(v-1)(v-1)})c_{v-1}(x) & \equiv \sum_{i=v}^{m-1} \alpha(x^{i(v-1)})c_i(x) \pmod{g_1(x)} \end{cases} \quad (70)$$

2) For $i = v - 1, v - 2, \dots, 1, 0$, do:

a) Compute composite message $f_i(x)$

$$f_i(x) \triangleq x^{r_{v-i}} a_i(x) + \pi^{(i)}(x) \cdot [c_{i+1}(x), c_{i+2}(x), \dots, c_{m-1}(x)]^T,$$

where $\pi^{(i)}(x)$ is given in (73).

b) Apply LFSR encoding with respect to $g_{v-i}(x)$

$$\mathcal{E}(\mathcal{U}_{r_{v-i}} f_i(x), g_{v-i}(x)) \rightarrow [\mathcal{U}_{r_{v-i}} f_i(x), p_i^*(x)].$$

c) Determine the parity polynomial $p_i(x)$ associated with $a_i(x)$

$$p_i(x) = p_i^*(x) + \mathcal{L}_{r_{v-i}} f_i(x).$$

We now measure the above computational complexity. LFSR encoding in Steps 1 and 2.(b) together takes $O(Rmn^2)$ XORing operations, where $R = \sum_{i=0}^{m-1} r_i/nm$ denotes the code rate. Running v rounds of Step 2.(a) take $O(vm \cdot Rn^2)$ XORing operations. Clearly, the algorithmic complexity is dominated by the execution of Step 2.(a), thus given by $O(Rvmn^2)$.

B. Hard Decoding Algorithm

The Berlekamp algorithm is a simplified version of the Berlekamp-Massey algorithm for decoding binary BCH codes by incorporating the special syndrome property

$$S_{2i+1} \triangleq y(\alpha^{2i+2}) = y(\alpha^{i+1})^2 = S_i^2, \quad i = 0, 1, 2, \dots$$

which yields zero discrepancies at even iterations of the Berlekamp-Massey algorithm [16]. In [17], the Berlekamp algorithm is reformulated neatly to perform t iterations. Similarly, we formulate the following Berlekamp updating algorithm to incrementally update the error locator polynomial utilizing the preceding results and the newly produced higher order syndromes.

Berlekamp Updating Algorithm

- Input: $[S_0, \dots, S_{2t-1}, \overbrace{S_{2t}, \dots, S_{2t'-1}}^{\text{new}}, \Lambda^{(2t)}(x), B^{(2t-1)}(x), L_{\Lambda}^{(2t)}, L_B^{(2t-1)}]$
- For $r = 2t, 2t + 2, \dots, 2t' - 2$, do:
 - Compute $\Delta^{(r+2)} = \sum_{i=0}^{L_{\Lambda}^{(r)}} \Lambda_i^{(r)} \cdot S_{r-i}$
 - Compute $\Lambda^{(r+2)}(x) = \Lambda^{(r)}(x) - \Delta^{(r+2)} \cdot x^2 B^{(r-1)}(x)$
 - If $\Delta^{(r+2)} \neq 0$ and $2L_{\Lambda}^{(r)} \leq r$, then
 - * Set $B^{(r+1)}(x) \leftarrow (\Delta^{(r+2)})^{-1} \cdot \Lambda^{(r)}(x)$
 - * Set $L_{\Lambda}^{(r+2)} \leftarrow L_B^{(r-1)} + 2, \quad L_B^{(r+1)} \leftarrow L_{\Lambda}^{(r)}$

– Else

- * Set $B^{(r+1)}(x) \leftarrow x^2 B^{(r-1)}(x)$
- * Set $L_B^{(r+1)} \leftarrow L_B^{(r-1)} + 2, \quad L_{\Lambda}^{(r+2)} \leftarrow L_{\Lambda}^{(r)}$

endif

endfor

- Output: $\Lambda^{(2t')}(x), B^{(2t'-1)}(x), L_{\Lambda}^{(2t')}, L_B^{(2t'-1)}$

The complete decoding algorithm differs slightly from that of GII-RS codes, as given below.

Decoding Algorithm for GII-BCH Codes

Input: $y(x) = [y_0(x), y_1(x), \dots, y_{m-1}(x)]$

- 1) Compute syndromes $\{S_{2j}^{(i)}\}_{j=0}^{t_0-1}$ and set $\{S_{2j+1}^{(i)}\}_{j=0}^{t_0-1} \leftarrow \{(S_j^{(i)})^2\}_{j=0}^{t_0-1}$, for $i = 0, 1, \dots, m-1$.
- 2) Apply the Berlekamp algorithm with input $\{S_j^{(i)}\}_{j=0}^{2t_0-1}$ to produce error locator polynomial set $(\Lambda^{(i,2t_0)}(x), B^{(i,2t_0-1)}(x))$, for $i = 0, 1, \dots, m-1$, respectively. For $i = 0, 1, \dots, m-1$, if $\Lambda^{(i,2t_0)}(x)$ has exactly $L_{\Lambda}^{(i,2t_0)}$ distinct roots within $\{\alpha^{-i}\}_{i=0}^{n-1}$, then correct the interleave y_i , else save the result $(\Lambda^{(i,2t_0)}(x), B^{(i,2t_0-1)}(x))$.
- 3) If all successful, then return the corrected codeword $c(x) = [c_0(x), c_1(x), \dots, c_{m-1}(x)]$; else if there are more than v uncorrectable interleaves, then declare failure; else denote by $I = \{l_1, l_2, \dots, l_b\}$ the index set of the uncorrectable interleaves.
- 4) Compute corrupted nested interleaves

$$\tilde{y}_i(x) = \sum_{j \in I} \alpha^{ij}(x) y_j(x) + \sum_{j \in I^c} \alpha^{ij}(x) c_j(x) \quad (76)$$
 for $i = 0, 1, 2, \dots, b-1$.
- 5) Compute higher order syndromes $\{\tilde{S}_{2j}^{(i)}\}_{j=t_0}^{t_{v-i}-1}$ over each nested interleaves $\tilde{y}_i(x)$, $i = 0, 1, 2, \dots, b-1$.
- 6) While $I \neq \emptyset$, do:
 - a) Compute syndromes $\{S_{2j}^{(l_i)}\}_{j=t_{v-b'}}^{t_{v-b}-1}$, $i = 1, 2, \dots, b$, through (77) for $j = t_{v-b'}, t_{v-b'} + 1, \dots, t_{v-b} - 1$, where b' is initialized to v .

Compute the corresponding odd-term syndromes

$$\begin{aligned} & [S_{2j+1}^{(l_1)}, S_{2j+1}^{(l_2)}, \dots, S_{2j+1}^{(l_b)}] \\ &= [(S_j^{(l_1)})^2, (S_j^{(l_2)})^2, \dots, (S_j^{(l_b)})^2] \end{aligned}$$

for $j = t_{v-b'}, t_{v-b'} + 1, \dots, t_{v-b} - 1$.

- b) Apply the Berlekamp updating algorithm with extra inputs $\{S_j^{(l_i)}\}_{j=t_{v-b'}}^{2t_{v-b}-1}$ to produce new error locator polynomial set $(\Lambda^{(l_i,2t_{v-b})}(x), B^{(l_i,2t_{v-b}-1)}(x))$, for $i = 1, 2, \dots, b$, respectively. For $i = 1, 2, \dots, b$, if $\Lambda^{(l_i,2t_{v-b})}(x)$ has exactly $L_{\Lambda}^{(l_i,2t_{v-b})}$ distinct roots within $\{\alpha^{-i}\}_{i=0}^{n-1}$, then correct the interleave y_{l_i} ; else save the result $(\Lambda^{(l_i,2t_{v-b})}(x), B^{(l_i,2t_{v-b}-1)}(x))$.

$$\begin{bmatrix} S_{2j}^{(l_1)} \\ S_{2j}^{(l_2)} \\ \vdots \\ S_{2j}^{(l_b)} \end{bmatrix} = \begin{bmatrix} 1 & 1 & \dots & 1 \\ \alpha^{(2j+1)l_1} & \alpha^{(2j+1)l_2} & \dots & \alpha^{(2j+1)l_b} \\ \vdots & \vdots & \ddots & \vdots \\ \alpha^{(2j+1)(b-1)l_1} & \alpha^{(2j+1)(b-1)l_2} & \dots & \alpha^{(2j+1)(b-1)l_b} \end{bmatrix}^{-1} \begin{bmatrix} \tilde{S}_{2j}^{(0)} \\ \tilde{S}_{2j}^{(1)} \\ \vdots \\ \tilde{S}_{2j}^{(b-1)} \end{bmatrix} \quad (77)$$

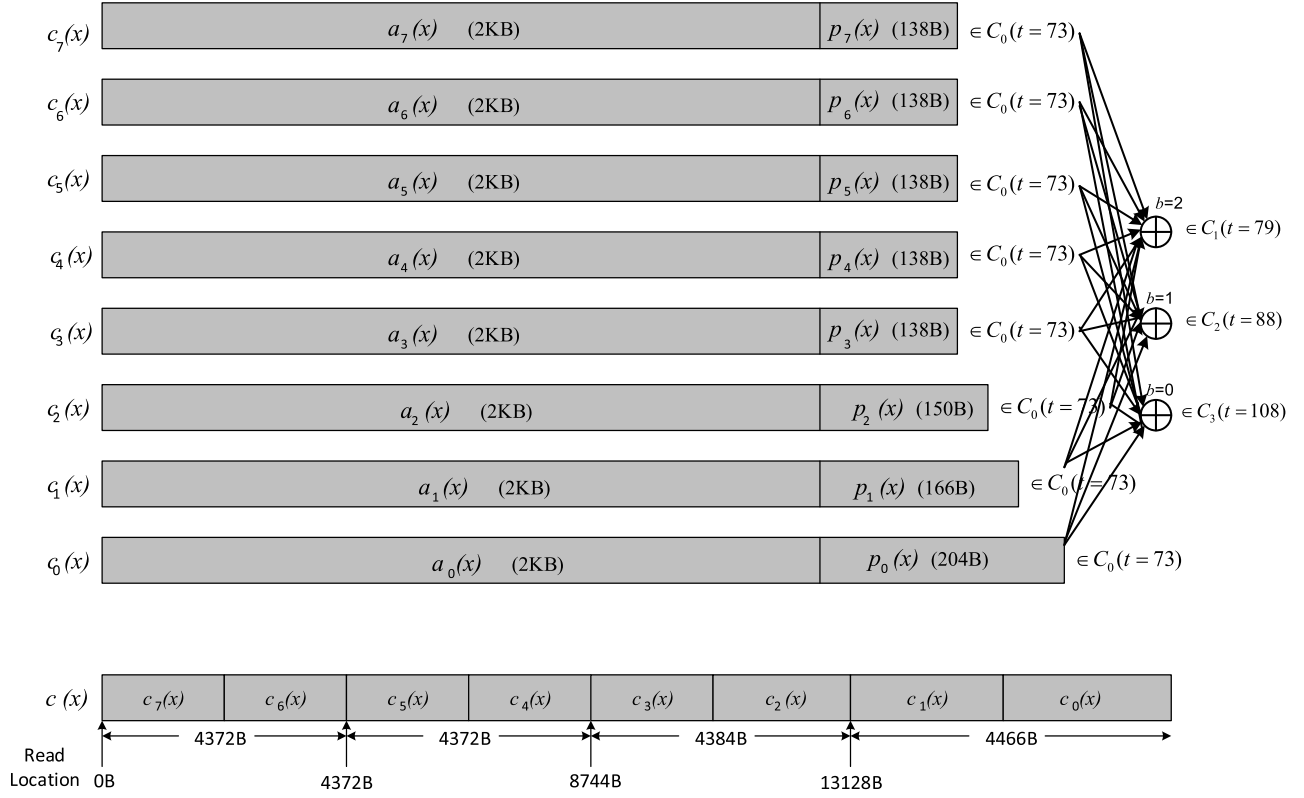


Fig. 6. An example of flash page programming using the proposed GII-BCH([8, 3]) code.

- c) Let I' be the index set of newly corrected interleaves. Set $I \leftarrow I - I'$, $b' \leftarrow b$, and $b \leftarrow |I|$. If $I' = \emptyset$ (no interleave is newly corrected) or $t_{v-b} = t_{v-b'}$ (it fails to produce extra higher order syndromes) then declare failure. If $I = \emptyset$ (all remaining interleaves are corrected) then return the corrected codeword.
- d) For each $i \in I'$, compute higher order syndromes $\{S_{2j}^{(i)}\}_{j=t_{v-b'}}^{t_v-1}$ through the recursive LFSR (21). Update higher order nested syndromes through

$$\left\{ \tilde{S}_{2j}^{(l)} \leftarrow \tilde{S}_{2j}^{(l-1)} - \sum_{i \in I'} \alpha^{(2j+1)il} S_{2j}^{(i)} \right\}_{j=t_{v-b'}}^{t_v-l-1}$$

for $0 \leq l < b$.

Note that the equality (77) is due to the fact: if $y'(x) = \alpha(x^l)y(x)$, then

$$S'_j \triangleq y'(\alpha^{j+1}) = \alpha^{l(j+1)}y(\alpha^{j+1}) = \alpha^{l(j+1)}S_j.$$

We now draw an example on the application of GII-BCH codes to flash memories. Currently, the prevalent design protects each 2K-Byte user data by a BCH code with t ranging from 60 to 100. Herein we assume $t = 80$. Note the code length lies between 2^{14} to 2^{15} , therefore, the code is defined in the field of $\text{GF}(2^{15})$. The length of parity overhead is $15 \times 80 = 1200$ bits. A 4K-Byte user data sector is divided into 2 BCH codewords and a 16K-Byte (with additional parity overhead) flash page contains 8 BCH

codewords. To enforce 4K-Byte data unit, each BCH interleave exactly encodes 2KB data with different parity length. This results in different interleave length. Luckily, GII-BCH coding allows for different interleave length without affecting the proposed encoder and decoder. The industry standard in data storage requests the sector failure rate below $1e-12$. Herein we enforce a stricter criterion such that the (flash) page failure rate is below $1e-12$. With simple interleaved 8 2K-Byte BCH codewords, it requires a raw bit-error-rate (RBER) of $1.7e-3$ to meet the requirement, whereas the simple interleaved scheme of 4 4K-Byte BCH codewords entails an RBER of $2.2e-3$ (see Table IV). We conducted exhaustive computer search to optimize the counterpart GII-BCH codes with different nesting nodes. The detailed results are also listed in the table. Note that the length of a user data sector is fixed to 4K-Byte, therefore, each interleave within a page varies slightly due to different parity length. Clearly, $v = m - 1 = 7$ achieves the highest RBER of $2.9e-3$. However, it also suffers the highest and non-negligible invocation rate of 0.234, that is, nearly one out of four reads fails by self-decoding and entails reading extra three sectors for nested decoding. Its random read throughput is degraded to $\frac{1}{1+0.234 \times 3} = 58.8\%$, which is unacceptable. In contrast, $v = 3$ achieves a superior trade-off. It enables to tolerate an RBER of $2.7e-3$, $\frac{2.7-1.7}{1.7} = 58.8\%$ higher than standard interleaved method, while suffering merely $\frac{3 \times 1.72e-3}{1+3 \times 1.72e-3} = 5.23e-3$ in random read throughput. Nevertheless, the II/GII scheme brings in undesirable attributes of variable interleave lengths and uneven read locations, which complicate the data management. Figure 6 depicts a pictorial

TABLE IV
COMPARISONS OF DIFFERENT NESTING SCHEMES TO REACH BELOW A FRAME-ERROR-RATE OF $1e-12$

Nesting Scheme	$d_{\min}$	p_b	P_f	P_i
Unnested(8, 2K-Byte, 161)	161	1.7e-3	6.95e-14	
Unnested(4, 4K-Byte, 301)	301	2.2e-3	8.12e-14	
GII([8, 1], 2K-Byte, [154, 203])	203	2.3e-3	2.88e-13	8.08e-7
GII([8, 2], 2K-Byte, [151, 171, 211])	211	2.5e-3	3.70e-14	5.68e-5
GII([8, 3], 2K-Byte, [147, 159, 177, 217])	217	2.7e-3	3.56e-13	1.72e-3
GII([8, 4], 2K-Byte, [143, 151, 163, 181, 221])	221	2.8e-3	6.85e-13	1.08e-2
GII([8, 5], 2K-Byte, [137, 145, 155, 165, 185, 227])	227	2.8e-3	1.20e-13	3.49e-2
GII([8, 6], 2K-Byte, [133, 139, 147, 155, 167, 185, 229])	229	2.8e-3	3.28e-14	7.07e-2
GII([8, 7], 2K-Byte, [129, 135, 141, 147, 157, 167, 185, 227])	227	2.9e-3	7.79e-13	2.34e-1

illustration of the proposed GII-BCH([8, 3]) scheme for flash page programming, where two-byte boundary truncation is assumed, and read is performed in 4K-Byte data sector unit.

IV. CONCLUSIONS

In this paper we systematically investigated generalized integrated interleaved (GII) coding scheme initially proposed by Tang and Koetter. We devised an efficient decoding algorithm by ignoring the first-layer miscorrection and by intelligently reusing preceding results during each iteration of a decoding attempt. Neglecting the first-layer miscorrection also enables to neatly formulate the decoding failure probability. We next derived an efficient erasure correcting algorithm under the context of RAID system recovery. We further constructed an algebraic systematic encoding algorithm, which had remained open. Analogously, we proposed a novel generalized integrated interleaved scheme over binary BCH codes, proved a lower bound on the minimum distance, and derived similar encoding and decoding algorithms as those of Reed-Solomon codes.

GII coding scheme is well suited for distributed environments where interleaves are distributed into different physical units and self-decoding is sufficient under normal operation condition. In addition, although equal length among interleaves is assumed throughout the context, it can be relaxed without affecting the proposed encoding and decoding algorithm. This is particularly useful for applications where equal data (but not code) length must be enforced among all interleaves. We note all existing GII decoding algorithms (including the proposed ones) are based on syndrome decoding for interleaves. To this extent, it is challenging to extend the GII coding scheme to general other codes. Finally, it is interesting to observe that, the Wu algorithm [17] is readily deployed for list decoding of GII RS/BCH codes, however, it is unclear how to apply the Guruswami-Sudan algorithm [18].

REFERENCES

- [1] A. M. Patel, "Two-level coding for error control in magnetic disk storage products," *IBM J. Res. Develop.*, vol. 33, no. 4, pp. 470–484, 1989.
- [2] K. A. S. Abdel-Ghaffar and M. Hassner, "Multilevel error-control codes for data storage channels," *IEEE Trans. Inf. Theory*, vol. 37, no. 3, pp. 735–741, May 1991.
- [3] C. E. Cox, M. A. Hassner, R. Koetter, and A. Patel, "Method and means for efficient error detection and correction in long byte strings using integrated interleaved Reed-Solomon codewords," U.S. Patent 5946328, Aug. 31, 1999.

- [4] M. A. Hassner, K. Abdel-Ghaffar, A. Patel, R. Koetter, and B. Trager, "Integrated interleaving—A novel ECC architecture," *IEEE Trans. Magn.*, vol. 37, no. 2, pp. 773–775, Mar. 2001.
- [5] Y. Wu, "A new encoding method for integrated-interleaved codes," in *Proc. IEEE Int. Symp. Inf. Theory*, Austin, TX, USA, Jun. 2010, pp. 983–987.
- [6] H. Asano, M. A. Hassner, N. N. Heise, and T. Tamura, "Multiple level, integrated sector format, error correction code encoding and decoding processes for data storage or communication devices and systems," U.S. Patent 6903887 B2, Jun. 7, 2005.
- [7] X. Tang and R. Koetter, "On the performance of integrated interleaving coding schemes," in *Proc. IEEE Int. Symp. Inf. Theory*, Chicago, IL, USA, Nov. 2004, p. 329.
- [8] X. Tang and R. Koetter, "A novel method for combining algebraic decoding and iterative processing," in *Proc. IEEE Int. Symp. Inf. Theory*, Seattle, WA, USA, Jul. 2006, pp. 474–478.
- [9] M. Blaum and S. Hetzler, (Jun. 2014). "Generalized concatenated types of codes for erasure correction." [Online]. Available: <https://arxiv.org/abs/1406.6270>
- [10] M. Blaum, J. L. Hafner, and S. R. Hetzler, "Nested multiple erasure correcting codes for storage arrays," U.S. Patent 8433979, Apr. 30, 2013.
- [11] E. L. Blokh and V. V. Zyablov, "Coding of generalized concatenated codes," *Problems Inf. Transmiss.*, vol. 10, no. 3, pp. 45–50, 1974.
- [12] M. Blaum, J. L. Hafner, and S. Hetzler, "Partial-MDS codes and their application to RAID type of architectures," *IEEE Trans. Inf. Theory*, vol. 59, no. 7, pp. 4510–4519, Jul. 2013.
- [13] J. Maucher, V. V. Zyablov, and M. Bossert, "On the equivalence of generalized concatenated codes and generalized error location codes," *IEEE Trans. Inf. Theory*, vol. 46, no. 2, pp. 642–649, Mar. 2000.
- [14] G. A. Gibson, *Redundant Disk Arrays*. New York, NY, USA: MIT Press, 1992.
- [15] R. J. McEliece and L. Swanson, "On the decoder error probability for Reed-Solomon codes," *IEEE Trans. Inf. Theory*, vol. 32, no. 5, pp. 701–703, Sep. 1986.
- [16] R. E. Blahut, *Algebraic Codes for Data Transmission*. Cambridge, U.K.: Cambridge Univ. Press, 2003.
- [17] Y. Wu, "New list decoding algorithms for Reed-Solomon and BCH codes," *IEEE Trans. Inf. Theory*, vol. 54, no. 8, pp. 3611–3630, Aug. 2008.
- [18] V. Guruswami and M. Sudan, "Improved decoding of Reed-Solomon codes and algebraic-geometry codes," *IEEE Trans. Inf. Theory*, vol. 45, no. 6, pp. 1757–1767, Sep. 1999.

Yingquan Wu (M'07–SM'08) was born in 1974, China. He received both the B.S. and M.S. degrees in mathematics from Harbin Institute of Technology, P.R. China in July 1996 and July 1997, respectively. In August 2000, he received the M.S. degree in electrical engineering at the State University of New York at Buffalo. In October 2004, he received the Ph.D. degree in electrical and computer engineering at the University of Illinois at Urbana-Champaign. Dr. Wu currently works in Micron Technology, Santa Clara, CA, where he is responsible for algorithms and architectures for error control coding and digital signal processing for solid-state drive. Dr. Wu's research interests lie in the areas of storage architecture, data compression, error control coding, and the high-speed VLSI design of general communication algorithms.